

Thadomal Shahani Engineering College

Bandra (W.), Mumbai- 400 050.

❧ CERTIFICATE ❧

Certify that Mr./Miss Aarav Malviya
of Computer Engineering Department, Semester VIII with
Roll No. 2103109 has completed a course of the necessary
experiments in the subject Applied Data Science under my
supervision in the **Thadomal Shahani Engineering College**
Laboratory in the year 2024 - 2025



Teacher In-Charge

Head of the Department

Date 09.04.25

Principal

EXPERIMENT 1

AIM: Explore the descriptive statistics on the given dataset

Theory:

Descriptive Statistics

Descriptive statistics summarize and describe the main features of a dataset. They provide a quick overview of the data and help in understanding its distribution, central tendency, and variability. Key measures include:

1. Measures of Central Tendency:
 - Mean: The average value of the dataset.
 - Median: The middle value when the data is sorted.
 - Mode: The most frequently occurring value.
2. Measures of Dispersion:
 - Range: The difference between the maximum and minimum values.
 - Variance: The average of the squared differences from the mean.
 - Standard Deviation: The square root of variance, indicating the spread of data.
 - Interquartile Range (IQR): The range between the first quartile (Q1) and the third quartile (Q3).
3. Shape of the Distribution:
 - Skewness: Measures the asymmetry of the data distribution.
 - Kurtosis: Measures the "tailedness" of the data distribution.
4. Other Measures:
 - Coefficient of Variation (CV): The ratio of the standard deviation to the mean, expressed as a percentage.
 - Trimmed Mean: The mean after removing a certain percentage of the smallest and largest values.
 - Sum of Squares: The sum of squared deviations from the mean.
5. Visualizations:
 - Box-and-Whisker Plot: Displays the distribution of data based on quartiles and identifies outliers.
 - Scatter Plot: Shows the relationship between two numeric variables.
 - Correlation Matrix: Displays the correlation coefficients between numeric variables.

Inferential Statistics

Inferential statistics make inferences about a population based on a sample of data. They help in testing hypotheses and drawing conclusions.

1. Distributions:
 - Normal Distribution: A symmetric, bell-shaped distribution where most values cluster around the mean.
 - Poisson Distribution: A discrete distribution that describes the probability of a given number of events occurring in a fixed interval.
2. Population Parameters and Sampling Errors:

- Population Parameters: Characteristics of the entire population (e.g., population mean, population variance).
 - Sampling Errors: Differences between the sample statistic and the population parameter.
3. Confidence Intervals:
- A range of values within which the population parameter is expected to lie, with a certain level of confidence (e.g., 95%).
4. Hypothesis Testing:
- Null Hypothesis (H0): A statement that there is no effect or no difference.
 - Alternative Hypothesis (H1): A statement that contradicts the null hypothesis.
 - Type I Error: Rejecting the null hypothesis when it is true (false positive).
 - Type II Error: Failing to reject the null hypothesis when it is false (false negative).
 - Z-Test: A statistical test used when the sample size is large and the population variance is known.
 - T-Test: A statistical test used when the sample size is small and the population variance is unknown.
 - ANOVA (Analysis of Variance): A test used to compare the means of three or more groups.

Code & Output:

```
#importing
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from scipy import stats
df = pd.read_csv("loan_data_set.csv")
print(df.head())
```

#output:

	Loan_ID	Gender	Married	Dependents	Education	Self_Employed
0	LP001002	Male	No	0	Graduate	No \
1	LP001003	Male	Yes	1	Graduate	No
2	LP001005	Male	Yes	0	Graduate	Yes
3	LP001006	Male	Yes	0	Not Graduate	No
4	LP001008	Male	No	0	Graduate	No

	ApplicantIncome	CoapplicantIncome	LoanAmount	Loan_Amount_Term
0	5849	0.0	NaN	360.0 \
1	4583	1508.0	128.0	360.0
2	3000	0.0	66.0	360.0
3	2583	2358.0	120.0	360.0
4	6000	0.0	141.0	360.0

	Credit_History	Property_Area	Loan_Status
0	1.0	Urban	Y
1	1.0	Rural	N
2	1.0	Urban	Y
3	1.0	Urban	Y
4	1.0	Urban	Y

A. DESCRIPTIVE STATISTICS

```
print("Descriptive Statistics:")
print(df.describe())
```

```
Descriptive Statistics:
```

	ApplicantIncome	CoapplicantIncome	LoanAmount	Loan_Amount_Term
count	614.000000	614.000000	592.000000	600.000000 \
mean	5403.459283	1621.245798	146.412162	342.000000
std	6109.041673	2926.248369	85.587325	65.12041
min	150.000000	0.000000	9.000000	12.000000
25%	2877.500000	0.000000	100.000000	360.000000
50%	3812.500000	1188.500000	128.000000	360.000000
75%	5795.000000	2297.250000	168.000000	360.000000
max	81000.000000	41667.000000	700.000000	480.000000

	Credit_History
count	564.000000
mean	0.842199
std	0.364878
min	0.000000
25%	1.000000
50%	1.000000
75%	1.000000
max	1.000000

```
print("\nMean:")
print(df.mean(numeric_only=True))

print("\nMedian:")
print(df.median(numeric_only=True))

print("\nMode:")
print(df.mode().iloc[0])
```

```
Mean:
ApplicantIncome      5403.459283
CoapplicantIncome     1621.245798
LoanAmount            146.412162
Loan_Amount_Term      342.000000
Credit_History        0.842199
dtype: float64
```

```
Median:
ApplicantIncome      3812.5
CoapplicantIncome     1188.5
LoanAmount            128.0
Loan_Amount_Term      360.0
Credit_History        1.0
dtype: float64
```

```
Mode:
Loan_ID              LP001002
Gender                Male
Married              Yes
Dependents            0
Education             Graduate
Self_Employed         No
ApplicantIncome       2500.0
CoapplicantIncome      0.0
LoanAmount            120.0
Loan_Amount_Term      360.0
Credit_History        1.0
Property_Area         Semiurban
Loan_Status            Y
Name: 0, dtype: object
```

```
print("\nMin:")
print(df.min(numeric_only=True))
```

```
print("\nMax:")
print(df.max(numeric_only=True))
```

```
print("\nSum:")
print(df.sum(numeric_only=True))
```



```
Min:
ApplicantIncome      150.0
CoapplicantIncome     0.0
LoanAmount           9.0
Loan_Amount_Term     12.0
Credit_History       0.0
dtype: float64
```

```
Max:
ApplicantIncome      81000.0
CoapplicantIncome    41667.0
LoanAmount          700.0
Loan_Amount_Term     480.0
Credit_History       1.0
dtype: float64
```

```
Sum:
ApplicantIncome      3.317724e+06
CoapplicantIncome    9.954449e+05
LoanAmount          8.667600e+04
Loan_Amount_Term     2.052000e+05
Credit_History      4.750000e+02
dtype: float64
```

```
print("\nRange:")
print(df.max(numeric_only=True) - df.min(numeric_only=True))
```

```
Range:
ApplicantIncome      80850.0
CoapplicantIncome    41667.0
LoanAmount          691.0
Loan_Amount_Term     468.0
Credit_History       1.0
dtype: float64
```

```
Q1 = df.quantile(0.25, numeric_only=True)
Q3 = df.quantile(0.75, numeric_only=True)
IQR = Q3 - Q1
print("\nFirst Quartile (Q1):")
print(Q1)
```

```
print("\nThird Quartile (Q3):")
print(Q3)
```

```
print("\nInterquartile Range (IQR):")
print(IQR)
```

First Quartile (Q1):

ApplicantIncome	2877.5
CoapplicantIncome	0.0
LoanAmount	100.0
Loan_Amount_Term	360.0
Credit_History	1.0

Name: 0.25, dtype: float64

Third Quartile (Q3):

ApplicantIncome	5795.00
CoapplicantIncome	2297.25
LoanAmount	168.00
Loan_Amount_Term	360.00
Credit_History	1.00

Name: 0.75, dtype: float64

Interquartile Range (IQR):

ApplicantIncome	2917.50
CoapplicantIncome	2297.25
LoanAmount	68.00
Loan_Amount_Term	0.00
Credit_History	0.00

dtype: float64

```
print("\nStandard Deviation:")
print(df.std(numeric_only=True))
```

Standard Deviation:

ApplicantIncome	6109.041673
CoapplicantIncome	2926.248369
LoanAmount	85.587325
Loan_Amount_Term	65.120410
Credit_History	0.364878

dtype: float64

```
variance = df.var(numeric_only=True)
```

```
print("Variance of Numeric Columns:")
print(variance)
```

Variance of Numeric Columns:

ApplicantIncome	3.732039e+07
CoapplicantIncome	8.562930e+06
LoanAmount	7.325190e+03
Loan_Amount_Term	4.240668e+03
Credit_History	1.331362e-01

dtype: float64

```
print("\nCorrelation Matrix:")
corr = df.corr(numeric_only=True)
print(corr)
```

```
# Standard Error of the Mean (SEM)
print("\nStandard Error of the Mean (SEM):")
print(df.sem(numeric_only=True))
```

Correlation Matrix:

	ApplicantIncome	CoapplicantIncome	LoanAmount
ApplicantIncome	1.000000	-0.116605	0.570909
CoapplicantIncome	-0.116605	1.000000	0.188619
LoanAmount	0.570909	0.188619	1.000000
Loan_Amount_Term	-0.045306	-0.059878	0.039447
Credit_History	-0.014715	-0.002056	-0.008433

	Loan_Amount_Term	Credit_History
ApplicantIncome	-0.045306	-0.014715
CoapplicantIncome	-0.059878	-0.002056
LoanAmount	0.039447	-0.008433
Loan_Amount_Term	1.000000	0.001470
Credit_History	0.001470	1.000000

Standard Error of the Mean (SEM):

ApplicantIncome	246.540857
CoapplicantIncome	118.093773
LoanAmount	3.517617
Loan_Amount_Term	2.658530
Credit_History	0.015364

dtype: float64

```
CV = lambda x: np.std(x) / np.mean(x) * 100
```

```
# Apply CV only to numeric columns
numeric_df = df.select_dtypes(include=[np.number]) # Select only
numeric columns
cv_results = numeric_df.apply(CV)
```

```
print("Coefficient of Variation:")
print(cv_results)
```

Coefficient of Variation:

ApplicantIncome	112.965871
CoapplicantIncome	180.346772
LoanAmount	58.407039
Loan_Amount_Term	19.025181
Credit_History	43.286074

dtype: float64

```
print("\nMissing Values:")
print(df.isnull().sum())
```

```
# Total Rows
print("\nTotal Rows (N total):")
print(len(df))
```



```
# Cumulative Sum
print("\nCummulative Sum:")
print(df.select_dtypes(include='number').cumsum())
```

```
# Percent and Cumulative Percent
print("\nPercent and Cumulative Percent:")
print(df.value_counts(normalize=True) * 100)
```

Missing Values:

```
Loan_ID      0
Gender       13
Married      3
Dependents   15
Education    0
Self_Employed 32
ApplicantIncome 0
CoapplicantIncome 0
LoanAmount   22
Loan_Amount_Term 14
Credit_History 50
Property_Area 0
Loan_Status  0
dtype: int64
```

Total Rows (N total):
614

Cummulative Sum:

	ApplicantIncome	CoapplicantIncome	LoanAmount	Loan_Amount_Term
0	5849	0.000000	NaN	360.0 \
1	10432	1508.000000	128.0	720.0
2	13432	1508.000000	194.0	1080.0
3	16015	3866.000000	314.0	1440.0
4	22015	3866.000000	455.0	1800.0
..	...	...	...	...
609	3293380	995204.919989	86063.0	203940.0
610	3297486	995204.919989	86103.0	204120.0
611	3305558	995444.919989	86356.0	204480.0
612	3313141	995444.919989	86543.0	204840.0
613	3317724	995444.919989	86676.0	205200.0

	Credit_History
0	1.0
1	2.0
2	3.0
3	4.0
4	5.0
..	...
609	472.0
610	473.0
611	474.0
612	475.0
613	475.0

[614 rows x 5 columns]

Percent and Cumulative Percent:

Loan_ID	Gender	Married	Dependents	Education	Self_Employed	ApplicantIncome	CoapplicantIncome	LoanAmount	Loan_Amount
LP001003	Male	Yes	1	Graduate	No	4583	1508.0	128.0	360.0
1.0		Rural	N	0.208333					
LP001005	Male	Yes	0	Graduate	Yes	3000	0.0	66.0	360.0
1.0		Urban	Y	0.208333					
LP002347	Male	Yes	0	Graduate	No	3246	1417.0	138.0	360.0
1.0		Semiurban	Y	0.208333					
LP002345	Male	Yes	0	Graduate	No	1025	2773.0	112.0	360.0
1.0		Rural	Y	0.208333					
LP002342	Male	Yes	2	Graduate	Yes	1600	20000.0	239.0	360.0
1.0		Urban	N	0.208333					
...									
LP001674	Male	Yes	1	Not Graduate	No	2600	2500.0	90.0	360.0
1.0		Semiurban	Y	0.208333					
LP001673	Male	No	0	Graduate	Yes	11000	0.0	83.0	360.0
1.0		Urban	N	0.208333					
LP001666	Male	No	0	Graduate	No	8333	3750.0	187.0	360.0
1.0		Rural	Y	0.208333					
LP001665	Male	Yes	1	Graduate	No	3125	2583.0	170.0	360.0
1.0		Semiurban	N	0.208333					

Trimmed Mean

```
print("\nTrimmed Mean (5%):")
```

```
print(stats.trim_mean(df['ApplicantIncome'], 0.05))
```

Sum of Squares

```
print("\nSum of Squares:")
```

```
print((df.select_dtypes(include='number') ** 2).sum())
```

Skewness

```
print("\nSkewness:")
```

```
print(df.skew(numeric_only=True))
```

Kurtosis

```
print("\nKurtosis:")
```

```
print(df.kurtosis(numeric_only=True))
```

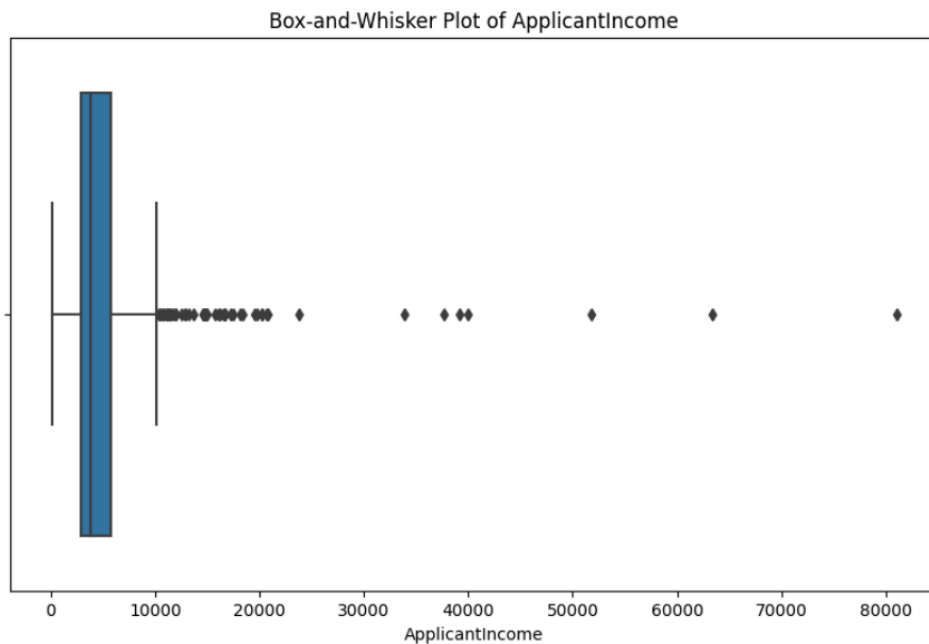
Trimmed Mean (5%):
4554.057761732852

Sum of Squares:
ApplicantIncome 4.080459e+10
CoapplicantIncome 6.862937e+09
LoanAmount 1.701961e+07
Loan_Amount_Term 7.271856e+07
Credit_History 4.750000e+02
dtype: float64

Skewness:
ApplicantIncome 6.539513
CoapplicantIncome 7.491531
LoanAmount 2.677552
Loan_Amount_Term -2.362414
Credit_History -1.882361
dtype: float64

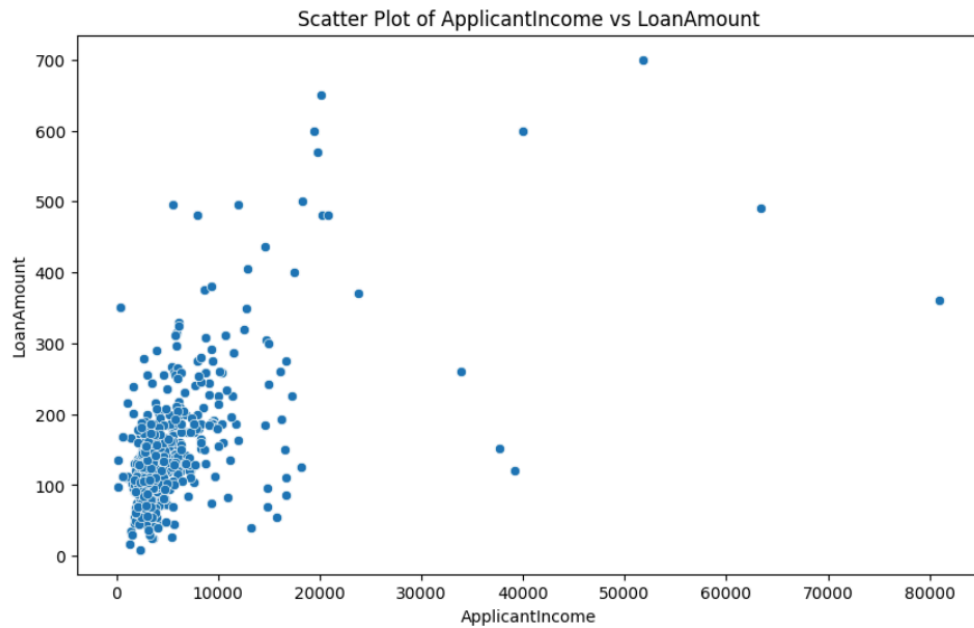
Kurtosis:
ApplicantIncome 60.540676
CoapplicantIncome 84.956384
LoanAmount 10.401533
Loan_Amount_Term 6.673474
Credit_History 1.548763
dtype: float64

```
# Box-and-Whisker Plot
plt.figure(figsize=(10, 6))
sns.boxplot(data=df, x="ApplicantIncome")
plt.title("Box-and-Whisker Plot of ApplicantIncome")
plt.show()
```

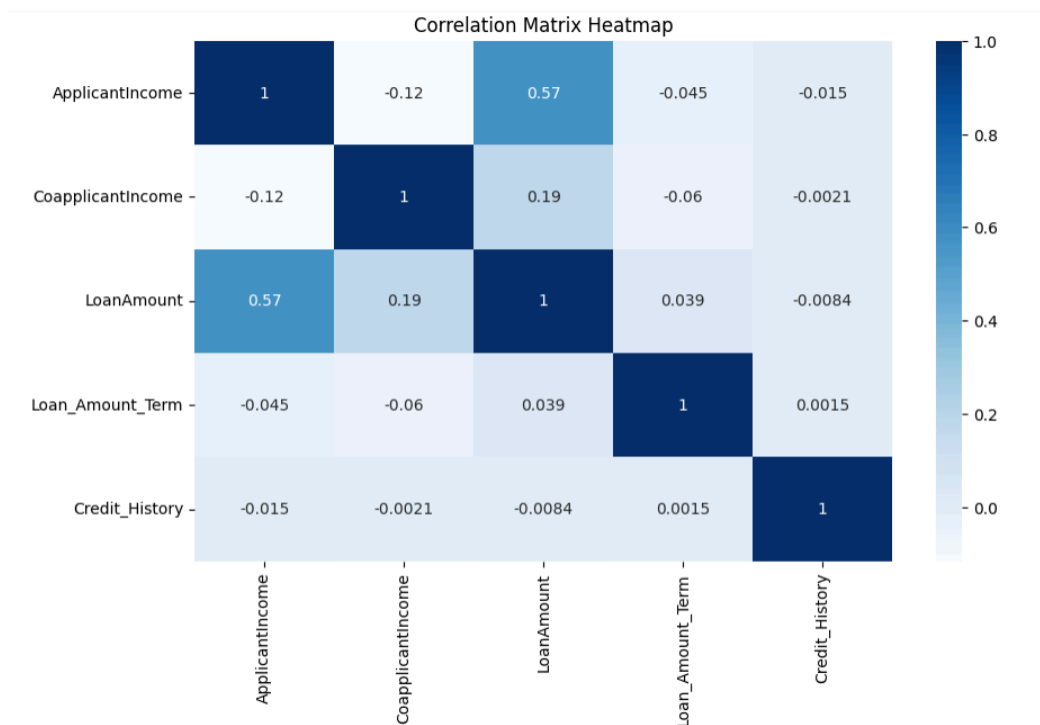


```
# Scatter Plot
plt.figure(figsize=(10, 6))
sns.scatterplot(data=df, x="ApplicantIncome", y="LoanAmount")
```

```
plt.title("Scatter Plot of ApplicantIncome vs LoanAmount")
plt.show()
```



```
# Correlation Matrix Heatmap
plt.figure(figsize=(10, 6))
sns.heatmap(corr, annot=True, cmap="Blues")
plt.title("Correlation Matrix Heatmap")
plt.show()
```



B. INFERENCE STATISTICS

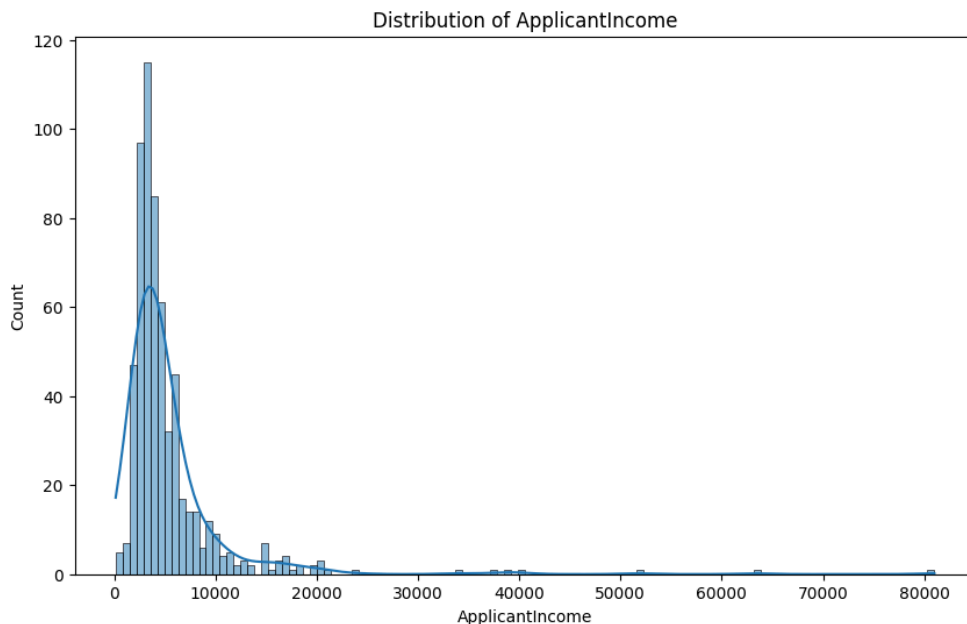
```
# Check for Normal Distribution
```



```
plt.figure(figsize=(10, 6))
sns.histplot(df['ApplicantIncome'], kde=True)
plt.title("Distribution of ApplicantIncome")
plt.show()

# Shapiro-Wilk Test for Normality
shapiro_test = stats.shapiro(df['ApplicantIncome'])
print("\nShapiro-Wilk Test for Normality:")
print(f"Test Statistic: {shapiro_test[0]}, p-value: {shapiro_test[1]}")

# Poisson Distribution (Example for LoanAmount)
poisson_test = stats.kstest(df['LoanAmount'], 'poisson',
args=(df['LoanAmount'].mean(),))
print("\nPoisson Distribution Test for LoanAmount:")
print(f"Test Statistic: {poisson_test[0]}, p-value: {poisson_test[1]}")
```



```
Shapiro-Wilk Test for Normality:
Test Statistic: 0.47126346826553345, p-value: 4.8126138406832944e-39

Poisson Distribution Test for LoanAmount:
Test Statistic: nan, p-value: nan
```

```
# Z-Test (Example: Compare ApplicantIncome mean to a population mean)
population_mean = 5000 # Hypothetical population mean
z_test = stats.zscore(df['ApplicantIncome'])
print("\nZ-Test Results:")
print(z_test)

# T-Test (Example: Compare ApplicantIncome for two groups)
group1 = df[df['Loan_Status'] == 'Y']['ApplicantIncome']
group2 = df[df['Loan_Status'] == 'N']['ApplicantIncome']
```

```

t_test = stats.ttest_ind(group1, group2)
print("\nT-Test Results:")
print(f"Test Statistic: {t_test[0]}, p-value: {t_test[1]}")

# ANOVA (Example: Compare ApplicantIncome across Property_Area
categories)
anova_test = stats.f_oneway(
    df[df['Property_Area'] == 'Urban']['ApplicantIncome'],
    df[df['Property_Area'] == 'Rural']['ApplicantIncome'],
    df[df['Property_Area'] == 'Semiurban']['ApplicantIncome']
)
print("\nANOVA Test Results:")
print(f"Test Statistic: {anova_test[0]}, p-value:
{anova_test[1]}")

```

Z-Test Results:

```

0      0.072991
1     -0.134412
2     -0.393747
3     -0.462062
4      0.097728

```

...

```

609   -0.410130
610   -0.212557
611    0.437174
612    0.357064
613   -0.134412

```

Name: ApplicantIncome, Length: 614, dtype: float64

T-Test Results:

Test Statistic: -0.11650844828724542, p-value: 0.907287812130518

ANOVA Test Results:

Test Statistic: 0.09280511472442918, p-value: 0.9113839401724976

EXPERIMENT 2

AIM: Apply Data Cleaning Techniques

Theory:

Imputation is the process of replacing missing or null values (like NaN or NA) in a dataset with estimated or calculated values. This helps in maintaining the completeness of the data for accurate analysis and modeling.

Imputation is important because:

Maintains Data Integrity: Ensures the dataset is complete for analysis.

Improves Model Performance: Most machine learning models can't handle missing values. Imputation helps in making accurate predictions.

Avoids Data Bias: Proper imputation prevents bias in the analysis caused by missing values.

Data Imputation Techniques:

1. Deletion of Rows with Missing Data

- **What It Is:** Remove rows containing any missing values.
- **When to Use:** When the number of missing values is very small.
- **Advantages:** Simple and easy to implement.
- **Disadvantages:** Can lead to data loss and bias if many rows are deleted.

2. Mean/Median Imputation

- **What It Is:** Replace missing values with the **mean or median** of the non-missing values in that column.
- **When to Use:** When the data is **numerical** and not heavily skewed.
- **Advantages:** Easy to implement and preserves the overall data distribution.
- **Disadvantages:** Can distort relationships between variables if the data is skewed.

3. Mode Imputation

- **What It Is:** Replace missing values with the **most frequent value** (mode) in the column.
- **When to Use:** For **categorical data**.
- **Advantages:** Simple and maintains the existing category distribution.
- **Disadvantages:** Can create over-representation of the mode, leading to biased results.

4. Arbitrary Value Imputation

- **What It Is:** Replace missing values with a fixed, **arbitrary value** (e.g., -999).
- **When to Use:** When you want to **flag missing values** with a distinct value.
- **Advantages:** Maintains dataset size and highlights missingness.
- **Disadvantages:** Can introduce outliers and affect model performance.

5. End of Tail Imputation

- **What It Is:** Replace missing values with **extreme values** (e.g., Mean + 3*Standard Deviation).
- **When to Use:** When missing values are likely to be extreme.

- **Advantages:** Highlights missingness as an outlier.
 - **Disadvantages:** Can distort statistical analysis and model predictions.
6. **Random Sample Imputation**
- **What It Is:** Replace missing values with **random samples** from the non-missing data in the column.
 - **When to Use:** When you want to maintain the existing distribution.
 - **Advantages:** Preserves the distribution and variability.
 - **Disadvantages:** Introduces randomness, which may reduce model stability.
7. **Frequent Category Imputation**
- **What It Is:** Replace missing values with the **most frequent category**.
 - **When to Use:** For categorical data with a dominant category.
 - **Advantages:** Maintains data consistency.
 - **Disadvantages:** May introduce bias by over-representing the frequent category.
8. **Adding a New Category as "Missing"**
- **What It Is:** Treat missing values as a **separate category** (e.g., "Unknown" or "Missing").
 - **When to Use:** For categorical data where missingness may have meaning.
 - **Advantages:** Captures the information about missingness.
 - **Disadvantages:** Increases the number of categories, possibly leading to model complexity.
9. **Regression Imputation**
- **What It Is:** Predicts missing values using a **regression model** based on other variables.
 - **When to Use:** When there is a strong correlation between variables.
 - **Advantages:** Leverages relationships between variables for accurate imputation.
 - **Disadvantages:** Can introduce bias if the regression model is not accurate.

Code & Output:

```
# Import necessary libraries
import pandas as pd
import numpy as np
from sklearn.impute import SimpleImputer
from sklearn.experimental import enable_iterative_imputer
from sklearn.impute import IterativeImputer
import seaborn as sns
import matplotlib.pyplot as plt
import random

# Load the dataset
df = pd.read_csv('loan_data_set.csv')

# Display basic information about the dataset
print("Dataset Overview:")
print(df.info())
```

```
print("\nFirst 5 rows of the dataset:\n", df.head())
```

Dataset Overview:

```
<class 'pandas.core.frame.DataFrame'>
```

```
RangeIndex: 614 entries, 0 to 613
```

```
Data columns (total 13 columns):
```

#	Column	Non-Null Count	Dtype
0	Loan_ID	614 non-null	object
1	Gender	601 non-null	object
2	Married	611 non-null	object
3	Dependents	599 non-null	object
4	Education	614 non-null	object
5	Self_Employed	582 non-null	object
6	ApplicantIncome	614 non-null	int64
7	CoapplicantIncome	614 non-null	float64
8	LoanAmount	592 non-null	float64
9	Loan_Amount_Term	600 non-null	float64
10	Credit_History	564 non-null	float64
11	Property_Area	614 non-null	object
12	Loan_Status	614 non-null	object

```
dtypes: float64(4), int64(1), object(8)
```

```
memory usage: 62.5+ KB
```

```
None
```

First 5 rows of the dataset:

	Loan_ID	Gender	Married	Dependents	Education	Self_Employed
0	LP001002	Male	No	0	Graduate	No
1	LP001003	Male	Yes	1	Graduate	No
2	LP001005	Male	Yes	0	Graduate	Yes
3	LP001006	Male	Yes	0	Not Graduate	No
4	LP001008	Male	No	0	Graduate	No

	ApplicantIncome	CoapplicantIncome	LoanAmount	Loan_Amount_Term
0	5849	0.0	NaN	360.0
1	4583	1508.0	128.0	360.0
2	3000	0.0	66.0	360.0
3	2583	2358.0	120.0	360.0
4	6000	0.0	141.0	360.0

	Credit_History	Property_Area	Loan_Status
0	1.0	Urban	Y
1	1.0	Rural	N
2	1.0	Urban	Y
3	1.0	Urban	Y
4	1.0	Urban	Y

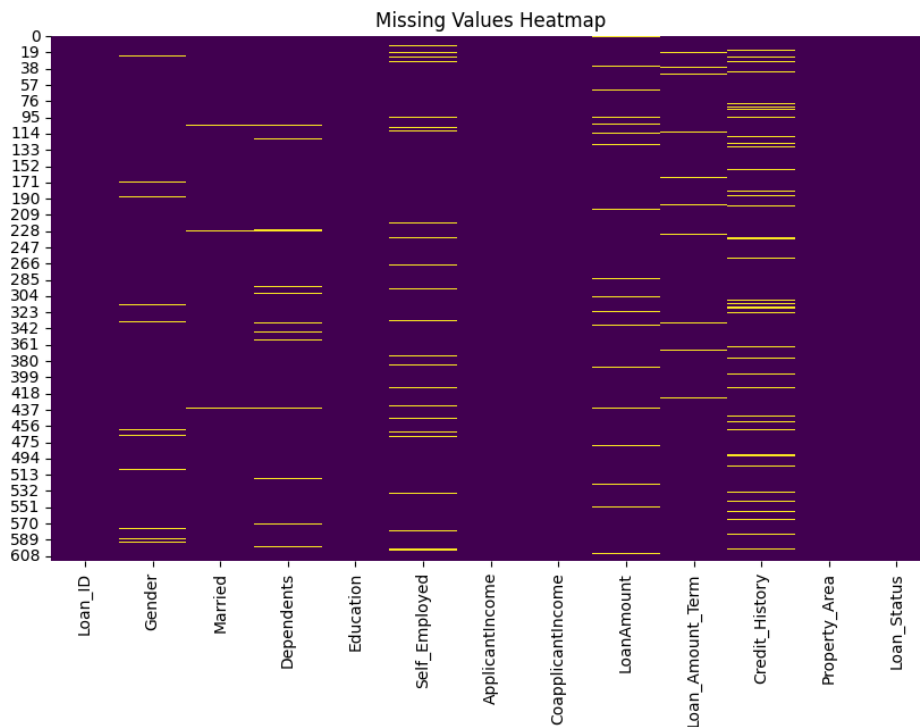
```
# Visualize missing values using Heatmap
```

```
plt.figure(figsize=(10, 6))
```

```
sns.heatmap(df.isnull(), cbar=False, cmap='viridis')
```

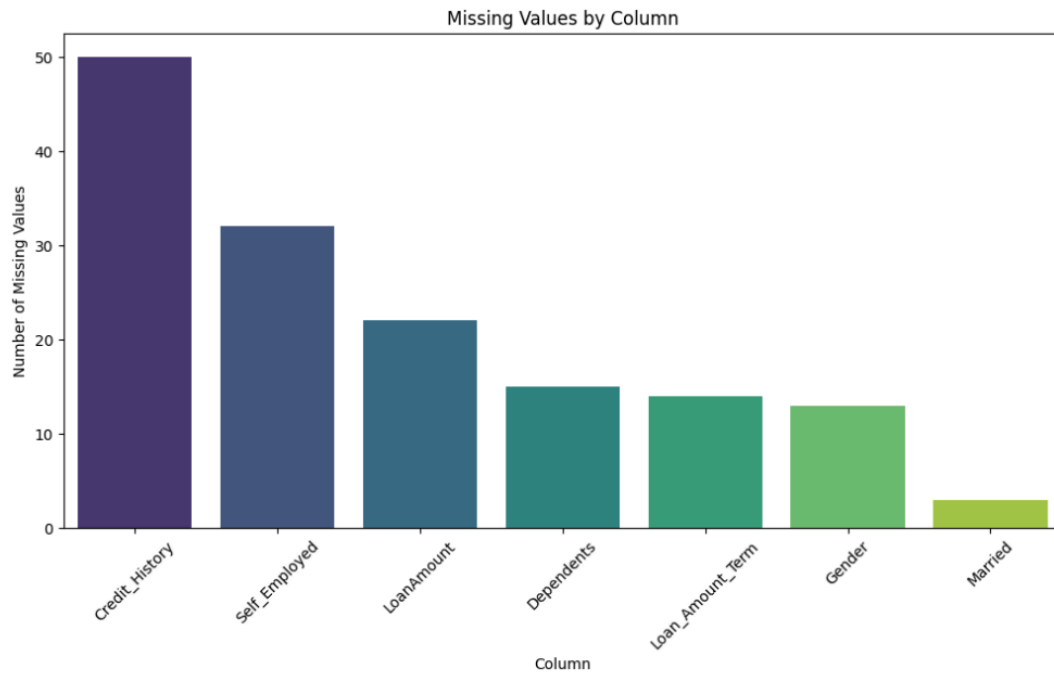
```
plt.title('Missing Values Heatmap')
```

```
plt.show()
```

```
# Visualize the count of missing values in each column using a bar
plot
missing_values = df.isnull().sum()
missing_values = missing_values[missing_values > 0]
missing_values.sort_values(ascending=False, inplace=True)

plt.figure(figsize=(12, 6))
sns.barplot(x=missing_values.index, y=missing_values.values,
palette='viridis')
plt.xticks(rotation=45)
plt.title('Missing Values by Column')
plt.xlabel('Column')
plt.ylabel('Number of Missing Values')
plt.show()
```



```
# Separate numeric and categorical columns
numeric_cols = df.select_dtypes(include=['float64',
'int64']).columns
categorical_cols = df.select_dtypes(include=['object']).columns
```

```
# 1. Deletion of rows with missing data
df_deletion = df.dropna()
print("\nAfter Deletion of Rows with Missing Data:\n",
df_deletion.head())
```

After Deletion of Rows with Missing Data:

	Loan_ID	Gender	Married	Dependents	Education	Self_Employed
1	LP001003	Male	Yes	1	Graduate	No
2	LP001005	Male	Yes	0	Graduate	Yes
3	LP001006	Male	Yes	0	Not Graduate	No
4	LP001008	Male	No	0	Graduate	No
5	LP001011	Male	Yes	2	Graduate	Yes

	ApplicantIncome	CoapplicantIncome	LoanAmount	Loan_Amount_Term
1	4583	1508.0	128.0	360.0
2	3000	0.0	66.0	360.0
3	2583	2358.0	120.0	360.0
4	6000	0.0	141.0	360.0
5	5417	4196.0	267.0	360.0

	Credit_History	Property_Area	Loan_Status
1	1.0	Rural	N
2	1.0	Urban	Y
3	1.0	Urban	Y
4	1.0	Urban	Y
5	1.0	Urban	Y

```
# 2. Mean/Median Imputation for numeric data
mean_imputer = SimpleImputer(strategy='mean')
median_imputer = SimpleImputer(strategy='median')
```

```

df_mean = df.copy()
df_median = df.copy()

df_mean[numeric_cols] =
mean_imputer.fit_transform(df[numeric_cols])
df_median[numeric_cols] =
median_imputer.fit_transform(df[numeric_cols])

print("\nAfter Mean Imputation:\n", df_mean.head())
print("\nAfter Median Imputation:\n", df_median.head())

# 3. Mode Imputation for categorical data
mode_imputer = SimpleImputer(strategy='most_frequent')
df_mode = df.copy()
df_mode[categorical_cols] =
mode_imputer.fit_transform(df[categorical_cols])

print("\nAfter Mode Imputation:\n", df_mode.head())

```

After Mean Imputation:

	Loan_ID	Gender	Married	Dependents	Education	Self_Employed
0	LP001002	Male	No	0	Graduate	No
1	LP001003	Male	Yes	1	Graduate	No
2	LP001005	Male	Yes	0	Graduate	Yes
3	LP001006	Male	Yes	0	Not Graduate	No
4	LP001008	Male	No	0	Graduate	No

	ApplicantIncome	CoapplicantIncome	LoanAmount	Loan_Amount_Term
0	5849.0	0.0	146.412162	360.0
1	4583.0	1508.0	128.000000	360.0
2	3000.0	0.0	66.000000	360.0
3	2583.0	2358.0	120.000000	360.0
4	6000.0	0.0	141.000000	360.0

	Credit_History	Property_Area	Loan_Status
0	1.0	Urban	Y
1	1.0	Rural	N
2	1.0	Urban	Y
3	1.0	Urban	Y
4	1.0	Urban	Y

After Median Imputation:

	Loan_ID	Gender	Married	Dependents	Education	Self_Employed
0	LP001002	Male	No	0	Graduate	No \
1	LP001003	Male	Yes	1	Graduate	No
2	LP001005	Male	Yes	0	Graduate	Yes
3	LP001006	Male	Yes	0	Not Graduate	No
4	LP001008	Male	No	0	Graduate	No

	ApplicantIncome	CoapplicantIncome	LoanAmount	Loan_Amount_Term
0	5849.0	0.0	128.0	360.0 \
1	4583.0	1508.0	128.0	360.0
2	3000.0	0.0	66.0	360.0
3	2583.0	2358.0	120.0	360.0
4	6000.0	0.0	141.0	360.0

	Credit_History	Property_Area	Loan_Status
0	1.0	Urban	Y
1	1.0	Rural	N
2	1.0	Urban	Y
3	1.0	Urban	Y
4	1.0	Urban	Y

After Mode Imputation:

	Loan_ID	Gender	Married	Dependents	Education	Self_Employed
0	LP001002	Male	No	0	Graduate	No \
1	LP001003	Male	Yes	1	Graduate	No
2	LP001005	Male	Yes	0	Graduate	Yes
3	LP001006	Male	Yes	0	Not Graduate	No
4	LP001008	Male	No	0	Graduate	No

	ApplicantIncome	CoapplicantIncome	LoanAmount	Loan_Amount_Term
0	5849	0.0	NaN	360.0 \
1	4583	1508.0	128.0	360.0
2	3000	0.0	66.0	360.0
3	2583	2358.0	120.0	360.0
4	6000	0.0	141.0	360.0

	Credit_History	Property_Area	Loan_Status
0	1.0	Urban	Y
1	1.0	Rural	N
2	1.0	Urban	Y
3	1.0	Urban	Y
4	1.0	Urban	Y

4. Arbitrary Value Imputation (e.g., -999 for numeric data)

```
df_arbitrary = df.copy()
df_arbitrary[numeric_cols] =
df_arbitrary[numeric_cols].fillna(-999)
```

```
print("\nAfter Arbitrary Value Imputation:\n",
df_arbitrary.head())
```

5. End of Tail Imputation (using mean + 3*std)

```
df_tail = df.copy()
for col in numeric_cols:
    end_of_tail_value = df[col].mean() + 3 * df[col].std()
    df_tail[col].fillna(end_of_tail_value, inplace=True)
```

```

print("\nAfter End of Tail Imputation:\n", df_tail.head())

# 6. Random Sample Imputation for numeric data
df_random = df.copy()
for col in numeric_cols:
    random_sample =
df[col].dropna().sample(df[col].isnull().sum(), random_state=42)
    random_sample.index = df[df[col].isnull()].index
    df_random.loc[df[col].isnull(), col] = random_sample

print("\nAfter Random Sample Imputation:\n", df_random.head())

```

After Arbitrary Value Imputation:

	Loan_ID	Gender	Married	Dependents	Education	Self_Employed
0	LP001002	Male	No	0	Graduate	No \
1	LP001003	Male	Yes	1	Graduate	No
2	LP001005	Male	Yes	0	Graduate	Yes
3	LP001006	Male	Yes	0	Not Graduate	No
4	LP001008	Male	No	0	Graduate	No

	ApplicantIncome	CoapplicantIncome	LoanAmount	Loan_Amount_Term
0	5849	0.0	-999.0	360.0 \
1	4583	1508.0	128.0	360.0
2	3000	0.0	66.0	360.0
3	2583	2358.0	120.0	360.0
4	6000	0.0	141.0	360.0

	Credit_History	Property_Area	Loan_Status
0	1.0	Urban	Y
1	1.0	Rural	N
2	1.0	Urban	Y
3	1.0	Urban	Y
4	1.0	Urban	Y

After End of Tail Imputation:

	Loan_ID	Gender	Married	Dependents	Education	Self_Employed
0	LP001002	Male	No	0	Graduate	No \
1	LP001003	Male	Yes	1	Graduate	No
2	LP001005	Male	Yes	0	Graduate	Yes
3	LP001006	Male	Yes	0	Not Graduate	No
4	LP001008	Male	No	0	Graduate	No

	ApplicantIncome	CoapplicantIncome	LoanAmount	Loan_Amount_Term
0	5849	0.0	403.174138	360.0 \
1	4583	1508.0	128.000000	360.0
2	3000	0.0	66.000000	360.0
3	2583	2358.0	120.000000	360.0
4	6000	0.0	141.000000	360.0

	Credit_History	Property_Area	Loan_Status
0	1.0	Urban	Y
1	1.0	Rural	N
2	1.0	Urban	Y
3	1.0	Urban	Y
4	1.0	Urban	Y

After Random Sample Imputation:

	Loan_ID	Gender	Married	Dependents	Education	Self_Employed
0	LP001002	Male	No	0	Graduate	No \
1	LP001003	Male	Yes	1	Graduate	No
2	LP001005	Male	Yes	0	Graduate	Yes
3	LP001006	Male	Yes	0	Not Graduate	No
4	LP001008	Male	No	0	Graduate	No

	ApplicantIncome	CoapplicantIncome	LoanAmount	Loan_Amount_Term
0	5849	0.0	123.0	360.0 \
1	4583	1508.0	128.0	360.0
2	3000	0.0	66.0	360.0
3	2583	2358.0	120.0	360.0
4	6000	0.0	141.0	360.0

	Credit_History	Property_Area	Loan_Status
0	1.0	Urban	Y
1	1.0	Rural	N
2	1.0	Urban	Y
3	1.0	Urban	Y
4	1.0	Urban	Y

7. Frequent Category Imputation for categorical data

```
df_frequent = df.copy()
for col in categorical_cols:
    most_frequent = df[col].mode()[0]
    df_frequent[col].fillna(most_frequent, inplace=True)
```

```
print("\nAfter Frequent Category Imputation:\n",
df_frequent.head())
```

8. Adding a New Category as "Missing" for categorical data

```
df_new_category = df.copy()
df_new_category[categorical_cols] =
df_new_category[categorical_cols].fillna("Missing")
```

```
print("\nAfter Adding New Category 'Missing':\n",
df_new_category.head())
```

9. Random Sample Imputation for categorical data

```
df_random_cat = df.copy()
for col in categorical_cols:
    random_sample =
df[col].dropna().sample(df[col].isnull().sum(), random_state=42)
    random_sample.index = df[df[col].isnull()].index
    df_random_cat.loc[df[col].isnull(), col] = random_sample
```

```
print("\nAfter Random Sample Imputation (Categorical):\n",
df_random_cat.head())
```

10. Regression Imputation using Iterative Imputer

```
iterative_imputer = IterativeImputer(random_state=42)
df_regression = df.copy()
```

```
df_regression[numeric_cols] =
iterative_imputer.fit_transform(df[numeric_cols])

print("\nAfter Regression Imputation:\n", df_regression.head())
```

After Frequent Category Imputation:

	Loan_ID	Gender	Married	Dependents	Education	Self_Employed
0	LP001002	Male	No	0	Graduate	No \
1	LP001003	Male	Yes	1	Graduate	No
2	LP001005	Male	Yes	0	Graduate	Yes
3	LP001006	Male	Yes	0	Not Graduate	No
4	LP001008	Male	No	0	Graduate	No

	ApplicantIncome	CoapplicantIncome	LoanAmount	Loan_Amount_Term
0	5849	0.0	NaN	360.0 \
1	4583	1508.0	128.0	360.0
2	3000	0.0	66.0	360.0
3	2583	2358.0	120.0	360.0
4	6000	0.0	141.0	360.0

	Credit_History	Property_Area	Loan_Status
0	1.0	Urban	Y
1	1.0	Rural	N
2	1.0	Urban	Y
3	1.0	Urban	Y
4	1.0	Urban	Y

After Adding New Category 'Missing':

	Loan_ID	Gender	Married	Dependents	Education	Self_Employed
0	LP001002	Male	No	0	Graduate	No \
1	LP001003	Male	Yes	1	Graduate	No
2	LP001005	Male	Yes	0	Graduate	Yes
3	LP001006	Male	Yes	0	Not Graduate	No
4	LP001008	Male	No	0	Graduate	No

	ApplicantIncome	CoapplicantIncome	LoanAmount	Loan_Amount_Term
0	5849	0.0	NaN	360.0 \
1	4583	1508.0	128.0	360.0
2	3000	0.0	66.0	360.0
3	2583	2358.0	120.0	360.0
4	6000	0.0	141.0	360.0

	Credit_History	Property_Area	Loan_Status
0	1.0	Urban	Y
1	1.0	Rural	N
2	1.0	Urban	Y
3	1.0	Urban	Y
4	1.0	Urban	Y

After Random Sample Imputation (Categorical):

	Loan_ID	Gender	Married	Dependents	Education	Self_Employed
0	LP001002	Male	No	0	Graduate	No \
1	LP001003	Male	Yes	1	Graduate	No
2	LP001005	Male	Yes	0	Graduate	Yes
3	LP001006	Male	Yes	0	Not Graduate	No
4	LP001008	Male	No	0	Graduate	No

	ApplicantIncome	CoapplicantIncome	LoanAmount	Loan_Amount_Term
0	5849	0.0	NaN	360.0 \
1	4583	1508.0	128.0	360.0
2	3000	0.0	66.0	360.0
3	2583	2358.0	120.0	360.0
4	6000	0.0	141.0	360.0

	Credit_History	Property_Area	Loan_Status
0	1.0	Urban	Y
1	1.0	Rural	N
2	1.0	Urban	Y
3	1.0	Urban	Y
4	1.0	Urban	Y

After Regression Imputation:

	Loan_ID	Gender	Married	Dependents	Education	Self_Employed
0	LP001002	Male	No	0	Graduate	No \
1	LP001003	Male	Yes	1	Graduate	No
2	LP001005	Male	Yes	0	Graduate	Yes
3	LP001006	Male	Yes	0	Not Graduate	No
4	LP001008	Male	No	0	Graduate	No

	ApplicantIncome	CoapplicantIncome	LoanAmount	Loan_Amount_Term
0	5849.0	0.0	138.159239	360.0 \
1	4583.0	1508.0	128.000000	360.0
2	3000.0	0.0	66.000000	360.0
3	2583.0	2358.0	120.000000	360.0
4	6000.0	0.0	141.000000	360.0

	Credit_History	Property_Area	Loan_Status
0	1.0	Urban	Y
1	1.0	Rural	N
2	1.0	Urban	Y
3	1.0	Urban	Y
4	1.0	Urban	Y

Visualization of the dataset after various imputation techniques

```
fig, axes = plt.subplots(3, 2, figsize=(15, 15))
```

```
fig.suptitle('Distribution of Imputed Data', fontsize=20)
```

```
sns.histplot(df_mean[numeric_cols[0]], ax=axes[0, 0],
color='blue')
```

```
axes[0, 0].set_title('Mean Imputation')
```

```
sns.histplot(df_median[numeric_cols[0]], ax=axes[0, 1],
color='green')
```

```
axes[0, 1].set_title('Median Imputation')
```

```
sns.histplot(df_arbitrary[numeric_cols[0]], ax=axes[1, 0],
color='red')
```

```

axes[1, 0].set_title('Arbitrary Value Imputation')

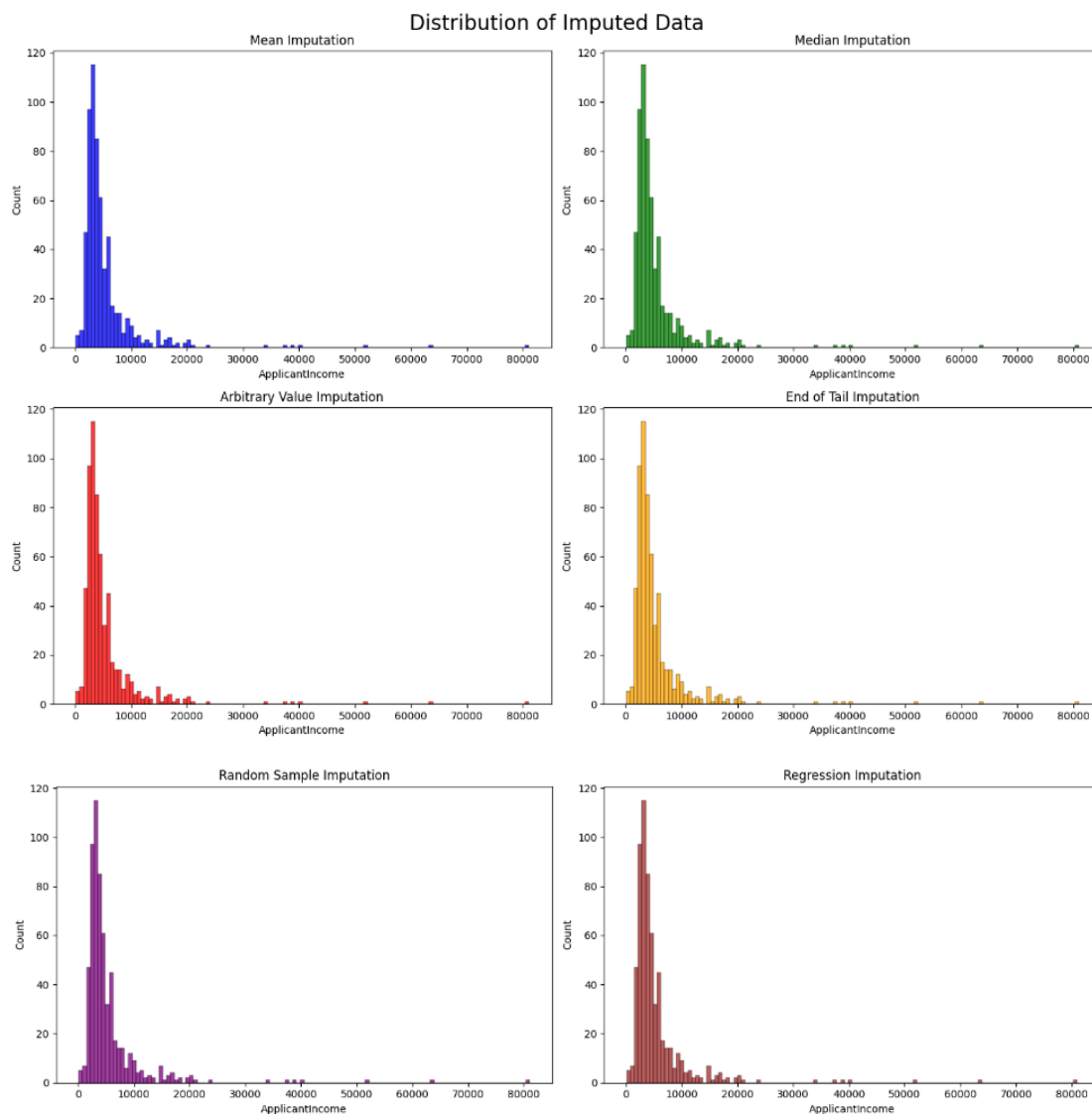
sns.histplot(df_tail[numeric_cols[0]], ax=axes[1, 1],
color='orange')
axes[1, 1].set_title('End of Tail Imputation')

sns.histplot(df_random[numeric_cols[0]], ax=axes[2, 0],
color='purple')
axes[2, 0].set_title('Random Sample Imputation')

sns.histplot(df_regression[numeric_cols[0]], ax=axes[2, 1],
color='brown')
axes[2, 1].set_title('Regression Imputation')

plt.tight_layout()
plt.show()

```



EXPERIMENT 3

AIM: Explore Data Visualization Techniques

Theory:

Importance of Data Visualization

What is the Purpose of Data Visualization?

Data visualization is the graphical representation of information and data using visual elements such as charts, graphs, and maps. The primary purpose of data visualization is to **simplify complex data** and make it more understandable, accessible, and useful for decision-making.

Key Purposes:

1. **Simplifies Complex Data** – Large datasets can be difficult to interpret in raw form. Visualizations help summarize and present data in an intuitive way.
2. **Enhances Understanding** – Patterns, trends, and correlations are easier to recognize when displayed visually.
3. **Improves Decision-Making** – Businesses and analysts can make informed decisions quickly based on visual insights.
4. **Identifies Patterns & Trends** – Helps in detecting trends, correlations, and outliers that may not be obvious in numerical data.
5. **Facilitates Communication** – Visual data representation is more effective for presentations and reports, making it easier to share insights with stakeholders.
6. **Increases Engagement** – Interactive and visually appealing dashboards keep users engaged and interested in the data.

Benefits of Data Visualization Tools

Data visualization tools help users create insightful visual representations of data with minimal effort. These tools come with built-in features for customization, interactivity, and analytics.

Key Benefits:

Faster Analysis – Converts raw data into meaningful visuals within seconds, making analysis quicker and more efficient.

Improved Accuracy – Reduces errors caused by manual data interpretation and helps in making more precise predictions.

Better Data Storytelling – Enables users to tell compelling stories through visuals that highlight important insights.

Enhanced Productivity – Saves time by automating data representation, allowing teams to focus on decision-making rather than data processing.

Real-time Data Monitoring – Many tools offer live dashboards that update dynamically, helping organizations track performance in real-time.

Customizable & Interactive – Users can filter, drill down, and explore data interactively for deeper insights.

Supports Multiple Data Sources – Most visualization tools integrate with databases, spreadsheets, APIs, and cloud storage for seamless data analysis.

Popular Data Visualization Tools:

- **Power BI** – Ideal for business intelligence and analytics.
- **Tableau** – Advanced analytics and interactive dashboards.
- **Matplotlib & Seaborn (Python)** – Great for scientific and statistical visualization.
- **Google Data Studio** – Free tool for creating interactive reports.
- **Excel** – Basic visualization for small datasets.

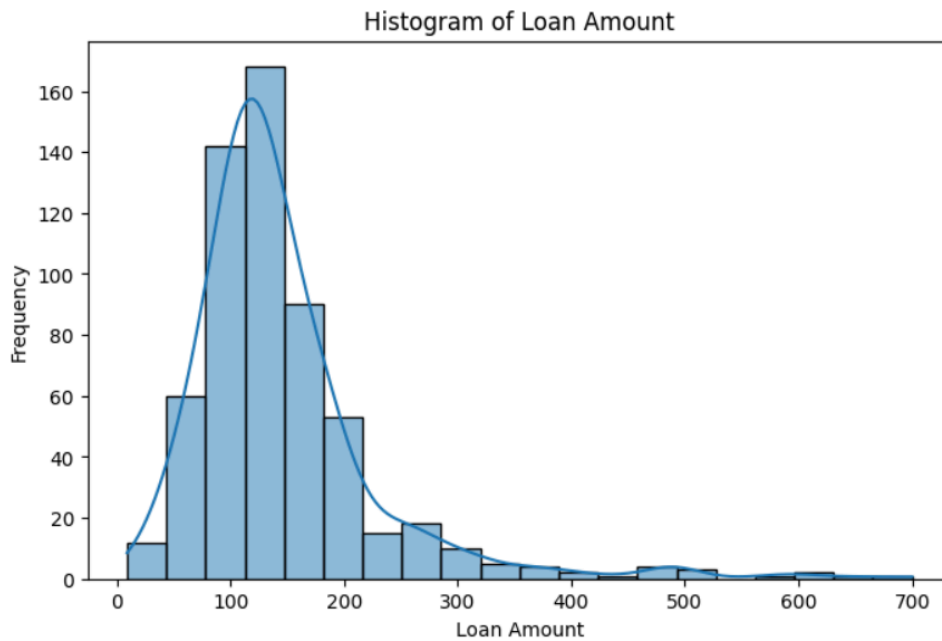
Code & Output:

```
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
import plotly.express as px
import plotly.graph_objects as go
from bokeh.plotting import figure, output_notebook, show
import missingno as msno
```

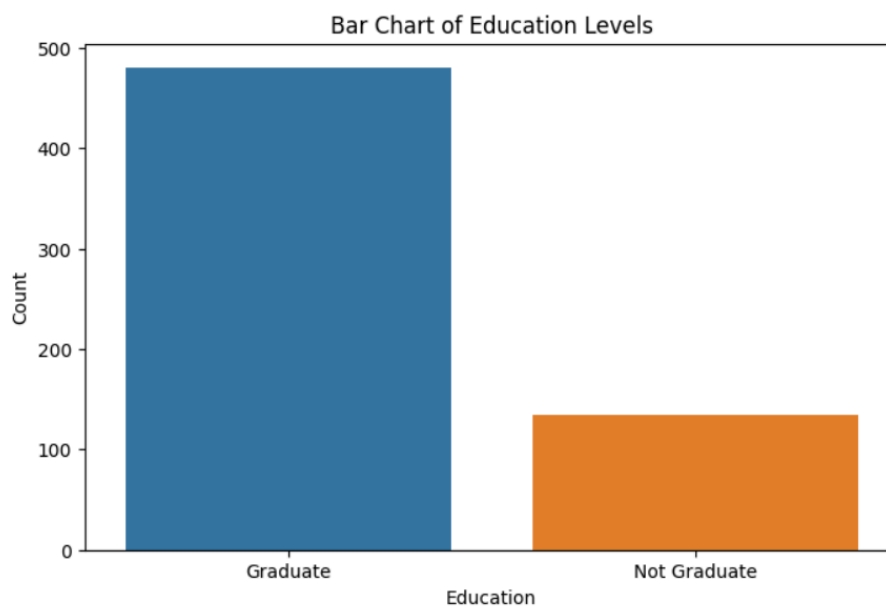
```
from pandas.plotting import scatter_matrix
# Load dataset
df = pd.read_csv("loan_data_set.csv")
# Display first few rows
df.head()
```

	Loan_ID	Gender	Married	Dependents	Education	Self_Employed	ApplicantIncome	CoapplicantIncome	LoanAmount	Loan_Amount_Term	Credit_History
0	LP001002	Male	No	0	Graduate	No	5849	0.0	NaN	360.0	1.0
1	LP001003	Male	Yes	1	Graduate	No	4583	1508.0	128.0	360.0	1.0
2	LP001005	Male	Yes	0	Graduate	Yes	3000	0.0	66.0	360.0	1.0
3	LP001006	Male	Yes	0	Not Graduate	No	2583	2358.0	120.0	360.0	1.0
4	LP001008	Male	No	0	Graduate	No	6000	0.0	141.0	360.0	1.0

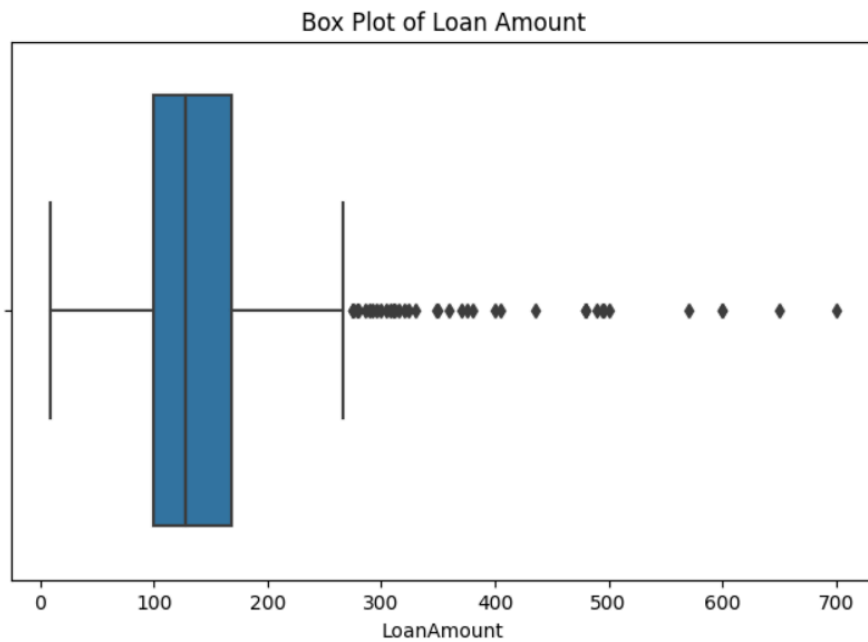
```
#Univariate Visualization
#Histogram
plt.figure(figsize=(8, 5))
sns.histplot(df['LoanAmount'], kde=True, bins=20)
plt.title("Histogram of Loan Amount")
plt.xlabel("Loan Amount")
plt.ylabel("Frequency")
plt.show()
```



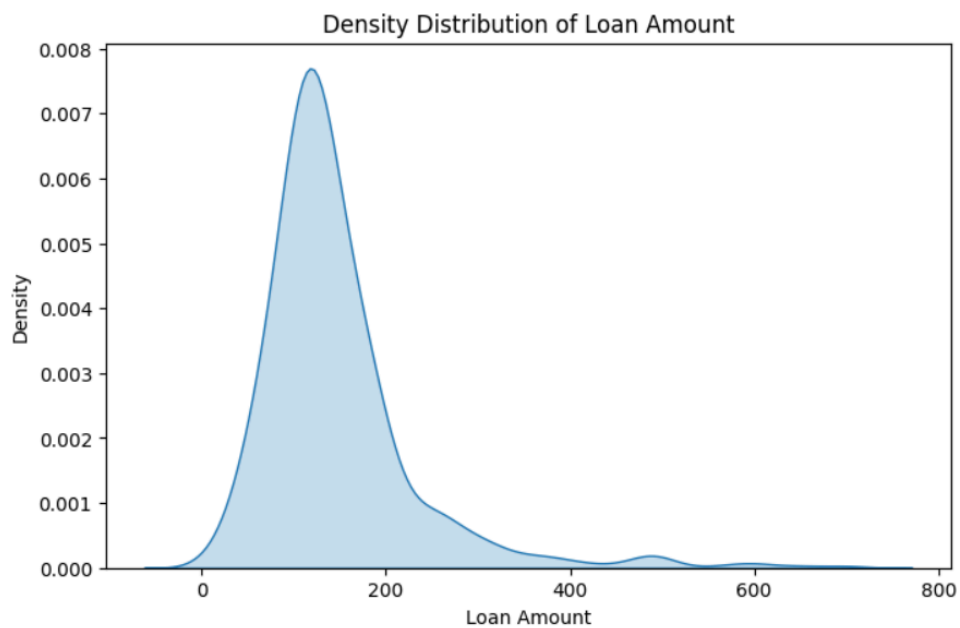
```
#BarChart
plt.figure(figsize=(8, 5))
sns.countplot(x="Education", data=df)
plt.title("Bar Chart of Education Levels")
plt.xlabel("Education")
plt.ylabel("Count")
plt.show()
```



```
#Quartile Plot(Box Plot)
plt.figure(figsize=(8, 5))
sns.boxplot(x=df["LoanAmount"])
plt.title("Box Plot of Loan Amount")
plt.show()
```

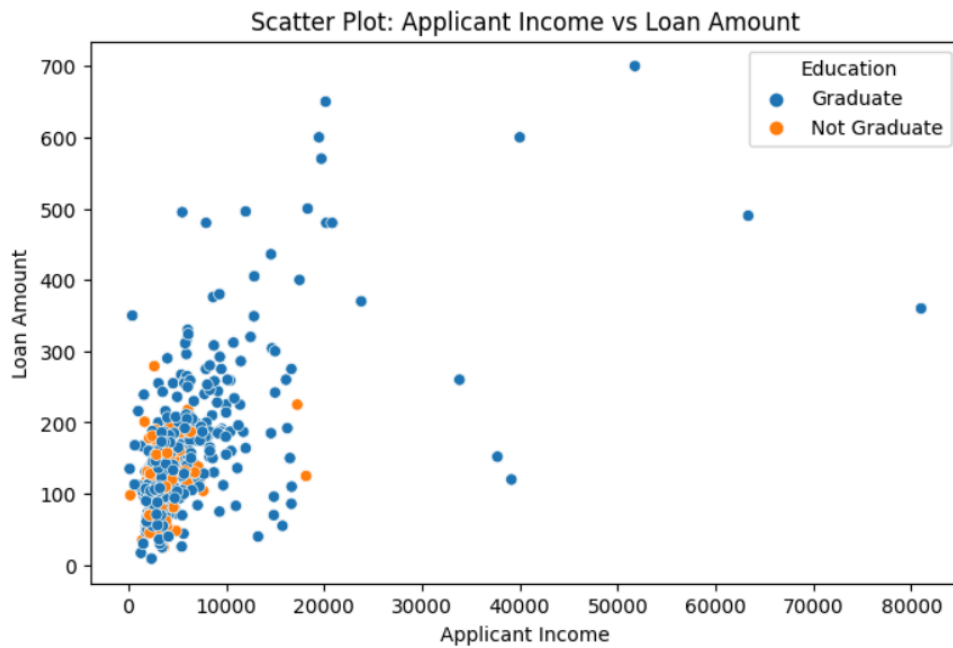


```
#Distribution Chart
plt.figure(figsize=(8, 5))
sns.kdeplot(df['LoanAmount'], shade=True)
plt.title("Density Distribution of Loan Amount")
plt.xlabel("Loan Amount")
plt.show()
```

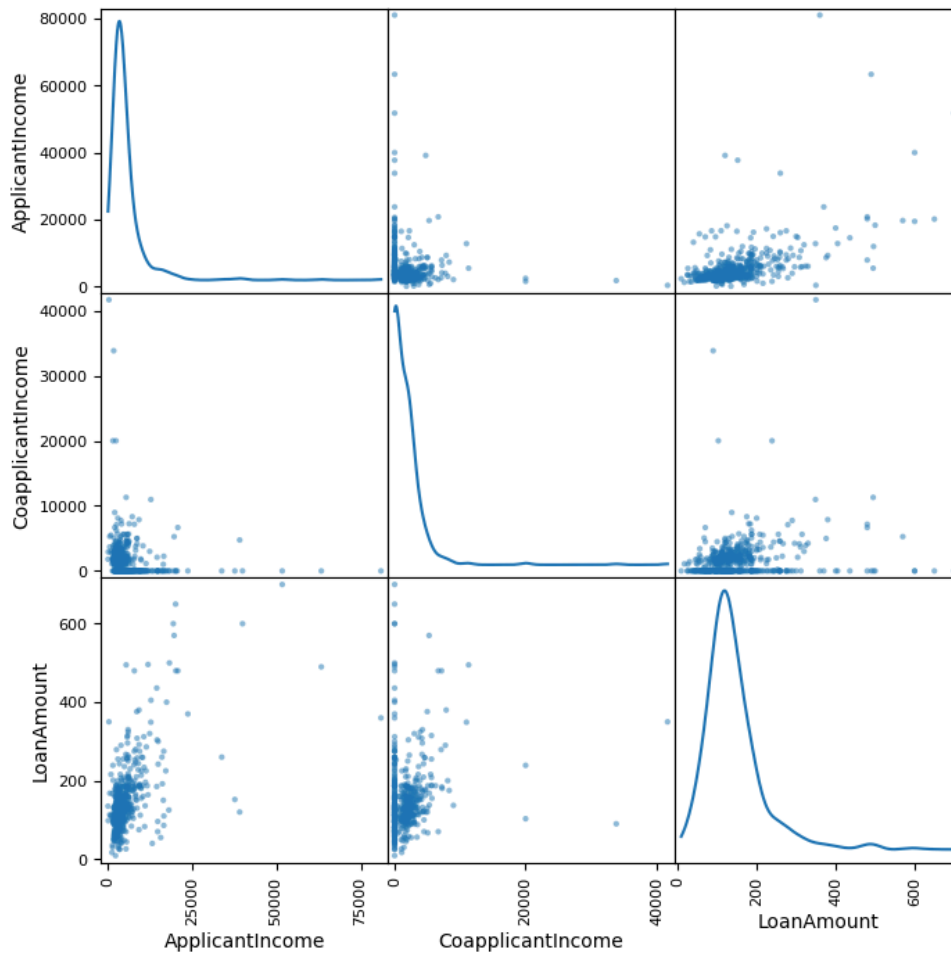


```
#Multivariate Visualization
#Scatter Plot
plt.figure(figsize=(8, 5))
sns.scatterplot(x='ApplicantIncome', y='LoanAmount',
hue='Education', data=df)
plt.title("Scatter Plot: Applicant Income vs Loan Amount")
plt.xlabel("Applicant Income")
```

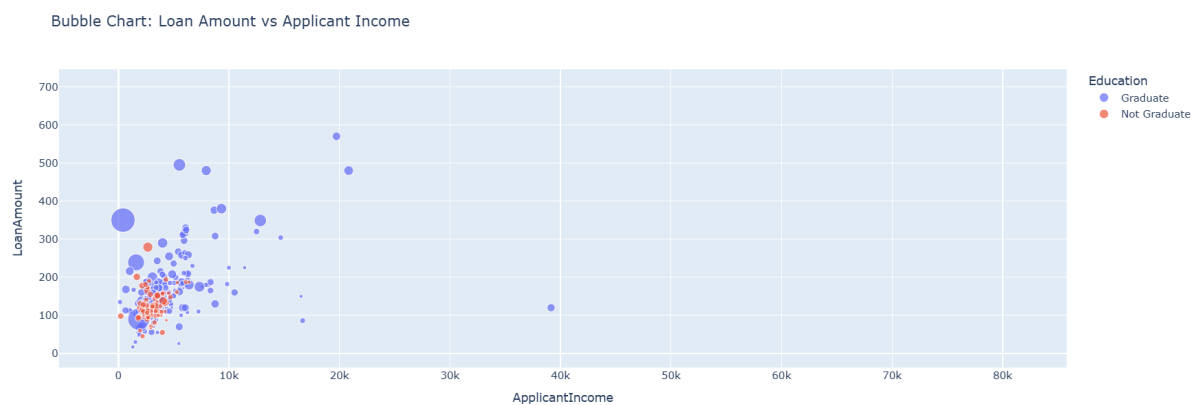
```
plt.ylabel("Loan Amount")
plt.show()
```



```
#Scatter Matrix
scatter_matrix(df[['ApplicantIncome', 'CoapplicantIncome',
'LoanAmount']], figsize=(8, 8), diagonal='kde')
plt.show()
```

```
#Bubble Chart
fig = px.scatter(df, x="ApplicantIncome", y="LoanAmount",
                 size="CoapplicantIncome", color="Education",
                 title="Bubble Chart: Loan Amount vs Applicant
Income")
fig.show()
```

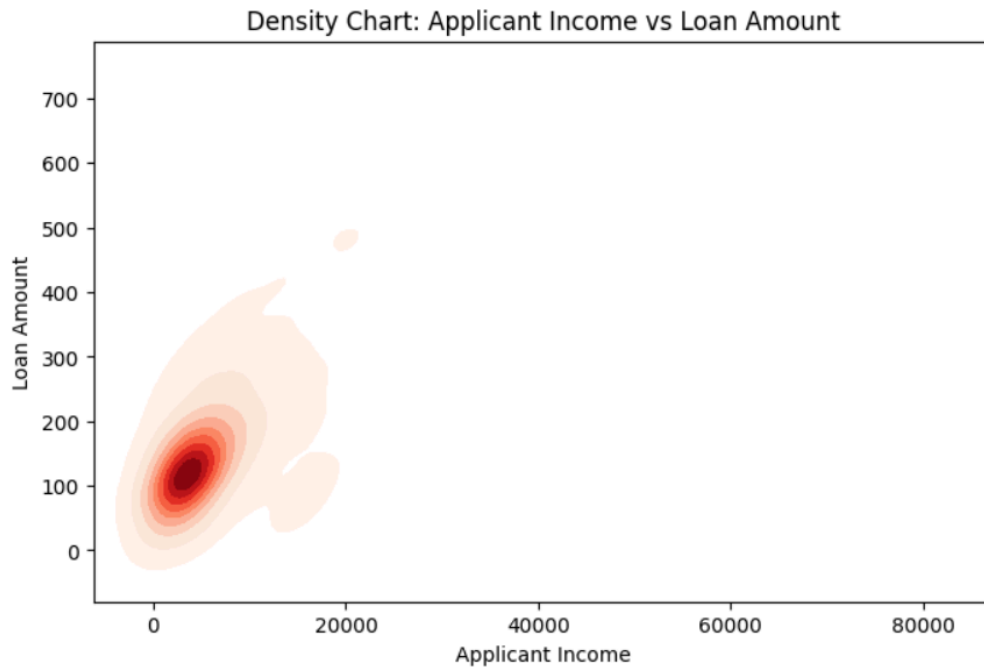


```
#density chart
plt.figure(figsize=(8, 5))
# Corrected kdeplot syntax
```

```

sns.kdeplot(x=df['ApplicantIncome'], y=df['LoanAmount'],
            cmap="Reds", fill=True)
plt.title("Density Chart: Applicant Income vs Loan Amount")
plt.xlabel("Applicant Income")
plt.ylabel("Loan Amount")
plt.show()

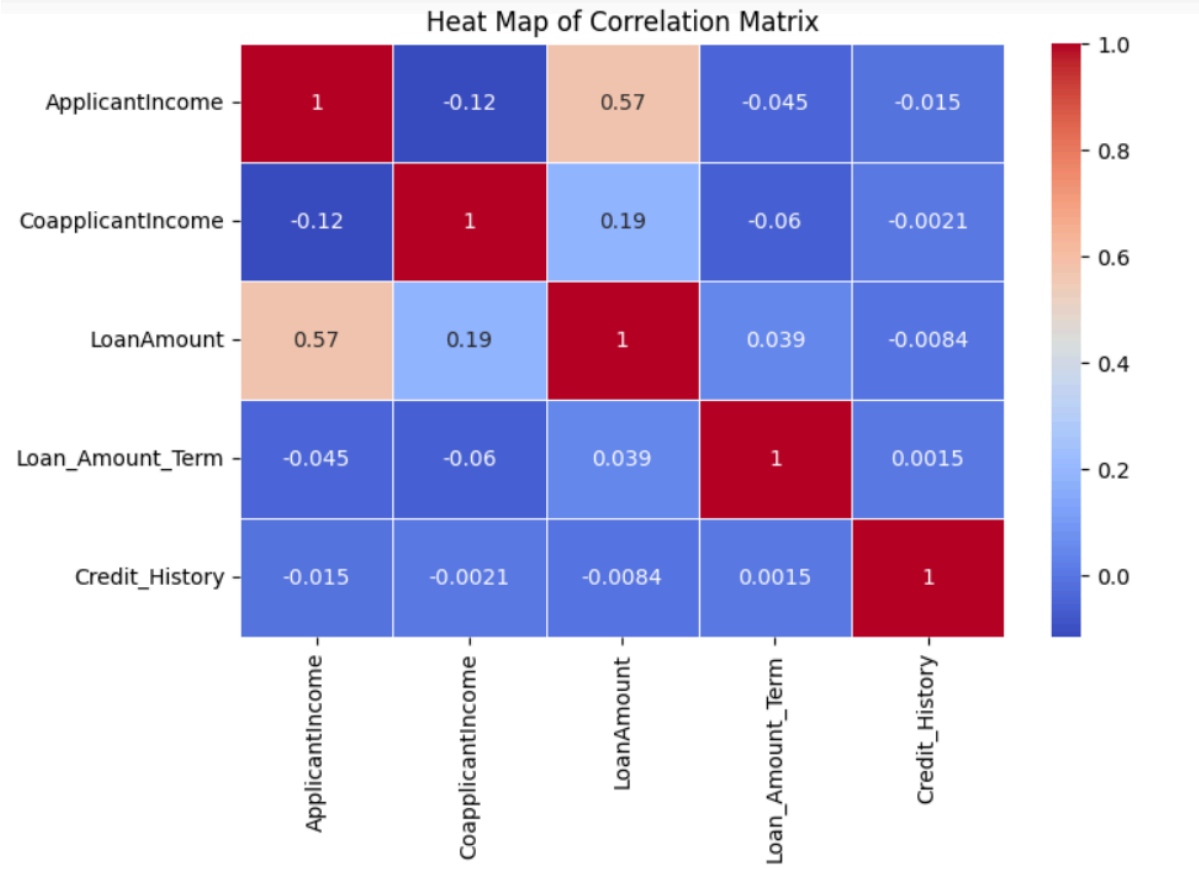
```



```

#Heat Map
# Select only numeric columns
numeric_df = df.select_dtypes(include=['number'])
plt.figure(figsize=(8, 5))
# Generate heatmap on numeric data only
sns.heatmap(numeric_df.corr(), annot=True, cmap="coolwarm",
            linewidths=0.5)
plt.title("Heat Map of Correlation Matrix")
plt.show()

```



EXPERIMENT 4

AIM: Implement and explore performance evaluation metrics for data models (Supervised)

Theory:

Supervised learning is a type of machine learning where a model is trained on a labeled dataset. The model learns patterns from the input data (features) and their corresponding correct outputs (labels). Once trained, the model makes predictions on unseen data.

To measure how well the model performs, we use different **evaluation metrics**, which vary depending on the type of supervised learning problem:

- **Classification** (predicting categories, e.g., spam detection)
- **Regression** (predicting continuous values, e.g., house prices)

Classification Metrics

Classification models predict **discrete categories** (e.g., spam detection, disease classification). The following metrics evaluate their performance:

1. Accuracy

Measures the proportion of correct predictions out of all predictions.

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

2. Precision (Positive Predictive Value - PPV)

Measures how many predicted positive cases are actually positive.

$$\text{Precision} = \frac{TP}{TP + FP}$$

3. Error Rate

Represents the proportion of incorrect predictions.

$$\text{Error Rate} = 1 - \text{Accuracy}$$

4. Sensitivity (Recall or True Positive Rate - TPR)

Measures how well the model identifies actual positive cases.

$$\text{Sensitivity} = \frac{TP}{TP + FN}$$

5. Specificity (True Negative Rate - TNR)

Measures how well the model identifies actual negative cases.

$$\text{Specificity} = \frac{TN}{TN + FP}$$

6. Receiver Operating Characteristic (ROC) Curve & AUC

- **ROC Curve:** A plot of **True Positive Rate (TPR)** vs. **False Positive Rate (FPR)**.
- **AUC (Area Under Curve):** Measures the model's ability to distinguish between classes.

$$\text{FPR} = \frac{FP}{FP + TN}$$

7. F1 Score

A harmonic mean of Precision and Recall, balancing both metrics.

$$F1 = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

8. Geometric Mean (G-Mean)

Measures the balance between **Sensitivity** and **Specificity**, ensuring both positive and negative cases are correctly classified.

$$\text{G-Mean} = \sqrt{\text{Sensitivity} \times \text{Specificity}}$$

Regression Metrics

Regression models predict **continuous values** (e.g., stock prices, temperature). The following metrics evaluate their performance:

1. Pearson Correlation Coefficient (rrr)

Measures the strength and direction of the relationship between actual and predicted values.

$$r = \frac{\sum (x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum (x_i - \bar{x})^2 \sum (y_i - \bar{y})^2}}$$

2. Coefficient of Determination (R² Score)

Indicates how much variance in the target variable is explained by the model.

$$R^2 = 1 - \frac{\sum (y_i - \hat{y}_i)^2}{\sum (y_i - \bar{y})^2}$$

3. Mean Squared Error (MSE)

The average squared differences between actual and predicted values.

$$MSE = \frac{1}{n} \sum (y_i - \hat{y}_i)^2$$

4. Root Mean Squared Error (RMSE)

The square root of MSE, measuring the standard deviation of errors.

$$RMSE = \sqrt{MSE}$$

5. Root Mean Squared Relative Error (RMSRE)

Similar to RMSE but considers relative errors.

$$RMSRE = \sqrt{\frac{1}{n} \sum \left(\frac{y_i - \hat{y}_i}{y_i} \right)^2}$$

6. Mean Absolute Error (MAE)

The average of absolute differences between actual and predicted values.

$$MAE = \frac{1}{n} \sum |y_i - \hat{y}_i|$$

7. Mean Absolute Percentage Error (MAPE)

Measures prediction accuracy as a percentage of actual values.

$$MAPE = \frac{100}{n} \sum \left| \frac{y_i - \hat{y}_i}{y_i} \right|$$

Code & Output:

```
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
import plotly.express as px
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression,
LinearRegression
from sklearn.metrics import accuracy_score, precision_score,
recall_score, f1_score, confusion_matrix, roc_curve, auc
from sklearn.metrics import mean_absolute_error,
mean_squared_error, r2_score

# Load dataset
df = pd.read_csv("supermarket_sales.csv") # Update the path if
required

# Display first few rows
print("Dataset Preview:\n", df.head())
```

Dataset Preview:

	Invoice ID	Branch	City	Customer type	Gender
0	750-67-8428	A	Yangon	Member	Female \
1	226-31-3081	C	Naypyitaw	Normal	Female
2	631-41-3108	A	Yangon	Normal	Male
3	123-19-1176	A	Yangon	Member	Male
4	373-73-7910	A	Yangon	Normal	Male

	Product line	Unit price	Quantity	Tax 5%	Total	Date
0	Health and beauty	74.69	7	26.1415	548.9715	1/5/2019 \
1	Electronic accessories	15.28	5	3.8200	80.2200	3/8/2019
2	Home and lifestyle	46.33	7	16.2155	340.5255	3/3/2019
3	Health and beauty	58.22	8	23.2880	489.0480	1/27/2019
4	Sports and travel	86.31	7	30.2085	634.3785	2/8/2019

	Time	Payment	cogs	gross margin percentage	gross income	Rating
0	13:08	Ewallet	522.83	4.761905	26.1415	9.1
1	10:29	Cash	76.40	4.761905	3.8200	9.6
2	13:23	Credit card	324.31	4.761905	16.2155	7.4
3	20:33	Ewallet	465.76	4.761905	23.2880	8.4
4	10:37	Ewallet	604.17	4.761905	30.2085	5.3

```

# Convert categorical target column into numeric (e.g., "Payment"
as classification label)
df['Payment'] = df['Payment'].astype('category').cat.codes #
Convert to numeric labels

# Feature selection for Classification
X_class = df[['Unit price', 'Quantity', 'Tax 5%', 'gross income']]
# Select numerical features
y_class = df['Payment'] # Classification Target

# Split classification data
X_train_class, X_test_class, y_train_class, y_test_class =
train_test_split(X_class, y_class, test_size=0.2, random_state=42)

# Train Logistic Regression Model
clf = LogisticRegression(max_iter=1000)
clf.fit(X_train_class, y_train_class)
y_pred_class = clf.predict(X_test_class)

# Classification Metrics Calculation
accuracy = accuracy_score(y_test_class, y_pred_class)
precision = precision_score(y_test_class, y_pred_class,
average='weighted')
recall = recall_score(y_test_class, y_pred_class,
average='weighted')
f1 = f1_score(y_test_class, y_pred_class, average='weighted')

# Confusion Matrix
conf_matrix = confusion_matrix(y_test_class, y_pred_class)

# ROC Curve
fpr, tpr, _ = roc_curve(y_test_class,
clf.predict_proba(X_test_class)[:, 1], pos_label=1)
roc_auc = auc(fpr, tpr)

# Print Classification Metrics
print("\n--- Classification Metrics ---")
print(f"Accuracy: {accuracy:.4f}")
print(f"Precision: {precision:.4f}")
print(f"Recall (Sensitivity): {recall:.4f}")
print(f"F1 Score: {f1:.4f}")

--- Classification Metrics ---
Accuracy: 0.2950
Precision: 0.2960
Recall (Sensitivity): 0.2950
F1 Score: 0.2859

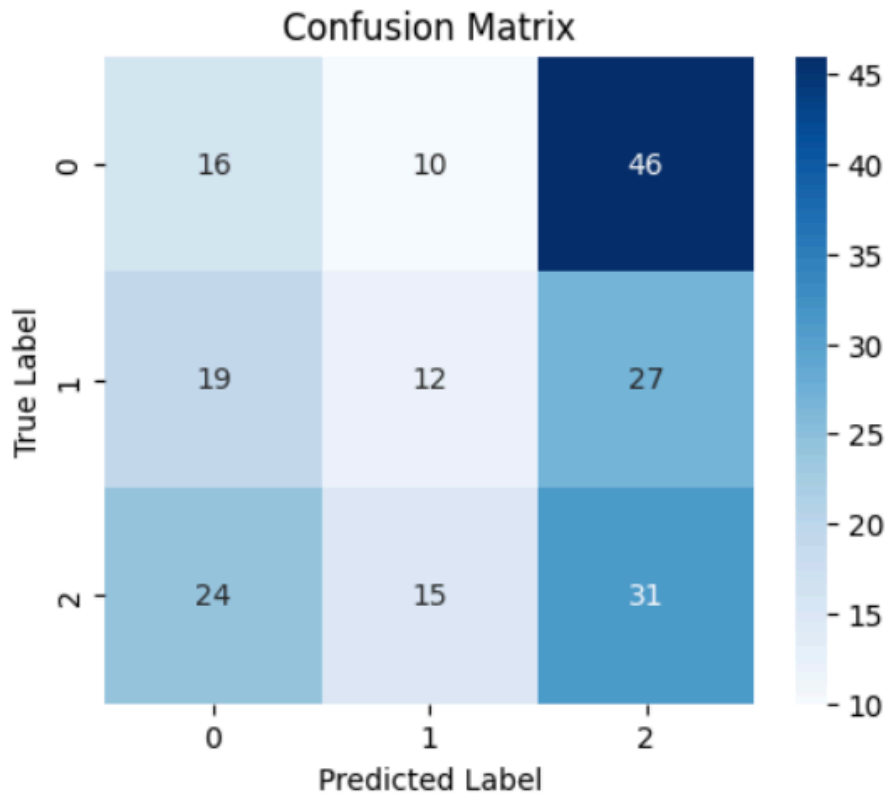
```



```

# Plot Confusion Matrix
plt.figure(figsize=(5,4))
sns.heatmap(conf_matrix, annot=True, fmt='d', cmap="Blues")
plt.xlabel("Predicted Label")
plt.ylabel("True Label")
plt.title("Confusion Matrix")
plt.show()

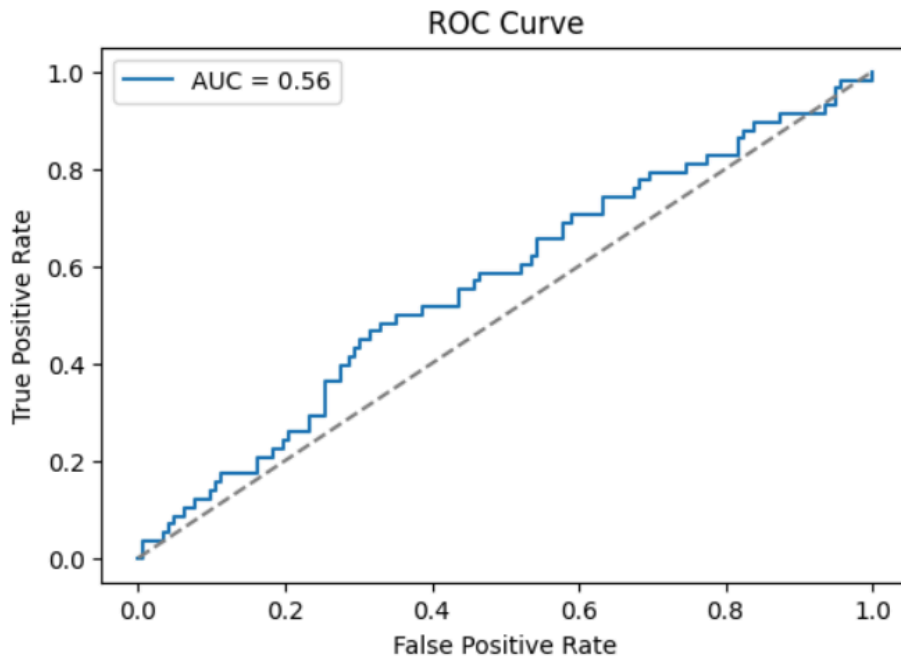
```



```

# Plot ROC Curve
plt.figure(figsize=(6,4))
plt.plot(fpr, tpr, label=f'AUC = {roc_auc:.2f}')
plt.plot([0, 1], [0, 1], linestyle="--", color="grey")
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.title("ROC Curve")
plt.legend()
plt.show()

```



```
# Feature selection for Regression
X_reg = df[['Unit price', 'Quantity', 'Tax 5%']] # Independent
Variables
y_reg = df['Total'] # Dependent Variable

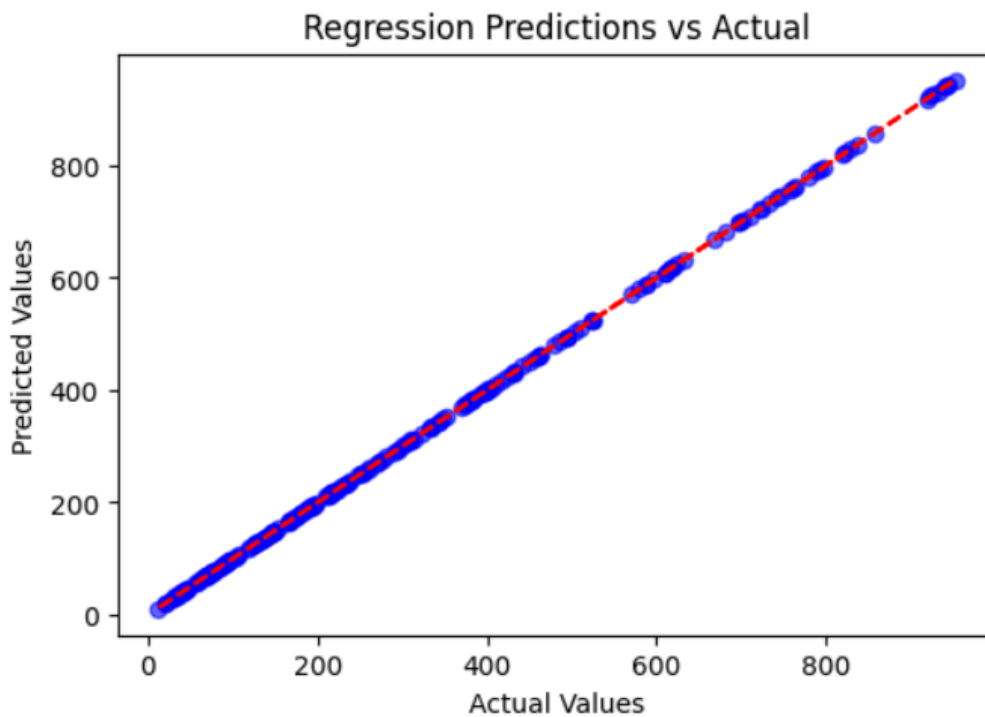
# Split regression data
X_train_reg, X_test_reg, y_train_reg, y_test_reg =
train_test_split(X_reg, y_reg, test_size=0.2, random_state=42)

# Train Linear Regression Model
reg = LinearRegression()
reg.fit(X_train_reg, y_train_reg)
y_pred_reg = reg.predict(X_test_reg)

# Regression Metrics Calculation
mae = mean_absolute_error(y_test_reg, y_pred_reg)
mse = mean_squared_error(y_test_reg, y_pred_reg)
rmse = np.sqrt(mse)
r2 = r2_score(y_test_reg, y_pred_reg)

# Print Regression Metrics
print("\n--- Regression Metrics ---")
print(f"Mean Absolute Error (MAE): {mae:.4f}")
print(f"Mean Squared Error (MSE): {mse:.4f}")
print(f"Root Mean Squared Error (RMSE): {rmse:.4f}")
print(f"R2 Score: {r2:.4f}")
```

```
--- Regression Metrics ---  
Mean Absolute Error (MAE): 0.0000  
Mean Squared Error (MSE): 0.0000  
Root Mean Squared Error (RMSE): 0.0000  
R2 Score: 1.0000  
  
# Plot Regression Predictions vs Actual  
plt.figure(figsize=(6,4))  
plt.scatter(y_test_reg, y_pred_reg, alpha=0.6, color="blue")  
plt.plot(y_test_reg, y_test_reg, color="red", linestyle="--") #  
Perfect fit line  
plt.xlabel("Actual Values")  
plt.ylabel("Predicted Values")  
plt.title("Regression Predictions vs Actual")  
plt.show()
```



EXPERIMENT 5

AIM: Implement and explore performance evaluation metrics for data models (Unsupervised Learning)

Theory:

Clustering is an unsupervised learning technique used to group similar data points based on certain features. Unlike classification, clustering does not rely on predefined labels, making evaluation more challenging. To assess the performance of clustering models, various internal and external evaluation metrics are used. Below are the key metrics used in clustering evaluation:

1. Rand Index (RI): The Rand Index (RI) measures the similarity between the predicted clustering assignments and the true class labels. It evaluates how well the clustering model has assigned data points by comparing pairs of samples.

$$\begin{aligned} E[RI] &= \frac{\text{Number of pairs in same cluster (actual)} \times \text{Number of pairs in same cluster (predicted)}}{\text{Total number of possible pairs}} \\ &= \frac{\sum_i \binom{a_i}{2} \times \sum_j \binom{b_j}{2}}{\binom{n}{2}} \end{aligned}$$

2. Adjusted Rand Index (ARI): The Adjusted Rand Index (ARI) is an improved version of the Rand Index that accounts for the probability of random clustering. It ensures that the score remains close to **zero for random assignments** and **one for perfect clustering**.

$$\begin{aligned} \text{Adjusted Rand Index (RI)} &= \frac{\text{Number of pair-wise true positive prediction} - E[RI]}{\text{Average number of pairs in same cluster for actual and predicted} - E[RI]} \\ &= \frac{\sum_{ij} \binom{n_{ij}}{2} - E[RI]}{\frac{1}{2} \sum_i \binom{a_i}{2} \times \sum_j \binom{b_j}{2} - E[RI]} \end{aligned}$$

3. Mutual Information (MI): Mutual Information (MI) measures the amount of shared information between the true labels and predicted clusters. It is based on entropy and evaluates how much knowing one variable (true labels) reduces uncertainty about the other variable (predicted clusters).

$$MI(U, V) = \sum_{i=1}^{|U|} \sum_{j=1}^{|V|} \frac{|U_i \cap V_j|}{N} \log \frac{N|U_i \cap V_j|}{|U_i||V_j|}$$

$|U_i|$ is the number of the samples in cluster U_i and $|V_j|$ is the number of the samples in cluster V_j

4. Silhouette Coefficient: The Silhouette Coefficient is an internal clustering metric that measures how similar a data point is to its own cluster compared to other clusters. It is based on the distances between data points.

$$S_i = \frac{b_i - a_i}{\max(a_i, b_i)}$$

Code:

```
import numpy as np
import pandas as pd
from itertools import combinations
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler
from collections import Counter
import math

# Load the Iris dataset from CSV
file_path = "Iris.csv" # Update this with the actual path if
needed
df = pd.read_csv(file_path)

# Drop the "Id" column if it exists (depends on the dataset
format)
if "Id" in df.columns:
    df = df.drop(columns=["Id"])

# Extract features and true labels
X = df.iloc[:, :-1].values # Features (Sepal & Petal dimensions)
y_true = df.iloc[:, -1].astype('category').cat.codes.values #
Convert species names to numeric labels

# Standardize the features
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# Apply KMeans Clustering
kmeans = KMeans(n_clusters=3, random_state=42, n_init=10)
y_pred = kmeans.fit_predict(X_scaled)

### **1. Implementing Rand Index (RI) and Adjusted Rand Index
(ARI)**
def rand_index(y_true, y_pred):
    pairs = list(combinations(range(len(y_true)), 2))
```

```

    a = sum((y_true[i] == y_true[j]) and (y_pred[i] == y_pred[j]))
for i, j in pairs)
    b = sum((y_true[i] != y_true[j]) and (y_pred[i] != y_pred[j]))
for i, j in pairs)
    return (a + b) / len(pairs)

```

```

def adjusted_rand_index(y_true, y_pred):
    n = len(y_true)
    clusters_true = Counter(y_true)
    clusters_pred = Counter(y_pred)

    sum_comb_c = sum(math.comb(v, 2) for v in
clusters_true.values()) # Combinations in actual clusters
    sum_comb_k = sum(math.comb(v, 2) for v in
clusters_pred.values()) # Combinations in predicted clusters
    sum_comb_ck = sum(math.comb(sum((y_true[i] == c and y_pred[i]
== k) for i in range(n)), 2)
                        for c in clusters_true for k in
clusters_pred)

    expected_index = (sum_comb_c * sum_comb_k) / math.comb(n, 2)
    max_index = 0.5 * (sum_comb_c + sum_comb_k)

    return (sum_comb_ck - expected_index) / (max_index -
expected_index)

```

2. Implementing Mutual Information (MI)

```

def mutual_information(y_true, y_pred):
    n = len(y_true)
    clusters_true = Counter(y_true)
    clusters_pred = Counter(y_pred)

    mi = 0
    for c in clusters_true:
        for k in clusters_pred:
            n_ck = sum((y_true[i] == c and y_pred[i] == k) for i
in range(n))
            if n_ck > 0:
                p_ck = n_ck / n
                p_c = clusters_true[c] / n
                p_k = clusters_pred[k] / n
                mi += p_ck * math.log(p_ck / (p_c * p_k))

    return mi

```

3. Implementing Silhouette Coefficient

```

def silhouette_coefficient(X, labels):
    n = len(X)

```

```

unique_clusters = set(labels)

def euclidean_distance(p1, p2):
    return np.linalg.norm(p1 - p2)

silhouette_scores = []

for i in range(n):
    same_cluster = [X[j] for j in range(n) if labels[j] ==
labels[i] and i != j]
    other_clusters = {c: [X[j] for j in range(n) if labels[j]
== c] for c in unique_clusters if c != labels[i]}

    if same_cluster:
        a_i = np.mean([euclidean_distance(X[i], p) for p in
same_cluster]) # Intra-cluster distance
    else:
        a_i = 0

    b_i = min(np.mean([euclidean_distance(X[i], p) for p in
cluster]) for cluster in other_clusters.values()) if
other_clusters else 0

    s_i = (b_i - a_i) / max(a_i, b_i) if max(a_i, b_i) > 0
else 0
    silhouette_scores.append(s_i)

return np.mean(silhouette_scores)

# Compute evaluation metrics
rand_idx = rand_index(y_true, y_pred)
adjusted_rand = adjusted_rand_index(y_true, y_pred)
mutual_info = mutual_information(y_true, y_pred)
silhouette_coeff = silhouette_coefficient(X_scaled, y_pred)

# Print results
print(f"Rand Index: {rand_idx:.4f}")
print(f"Adjusted Rand Index: {adjusted_rand:.4f}")
print(f"Mutual Information: {mutual_info:.4f}")
print(f"Silhouette Coefficient: {silhouette_coeff:.4f}")

```

Output:

Rand Index: 0.8322

Adjusted Rand Index: 0.6201

Mutual Information: 0.7241

Silhouette Coefficient: 0.4590

EXPERIMENT 6

AIM: Implement time series forecasting

Theory:

Time series analysis and forecasting are essential for understanding patterns in data collected over time. By studying historical trends, businesses and researchers can make informed decisions about future events. The key reasons for conducting time series analysis and forecasting include:

1. Differentiating Between Random Fluctuations and Outliers

Time series data often contains short-term irregular variations that can mislead decision-making. By analyzing the data, we can determine whether a sudden change is a natural fluctuation or an outlier. Outliers are extreme values that deviate significantly from the trend and may be caused by unexpected events, errors in data collection, or external influences. Identifying outliers helps in making more accurate predictions and avoiding misleading conclusions.

2. Separating Genuine Insights from Seasonal Variations

Many datasets exhibit seasonality, meaning they follow a recurring pattern over specific time intervals (e.g., daily, monthly, yearly). For example, sales of winter clothing increase in colder months and decline in summer. Without time series analysis, it can be difficult to distinguish whether a trend is genuine growth or just a seasonal effect. By decomposing the data into trend, seasonal, and residual components, we can better understand the underlying behavior and make accurate forecasts.

3. Understanding How Data Changes Over Time

Time series analysis helps in recognizing how a variable evolves over different periods. For instance, stock prices, temperature variations, or economic indicators change continuously, and analyzing past patterns helps predict future movements. By examining trends, seasonality, and irregular components, we can determine whether a process is stable, increasing, or declining over time, which aids in better planning and decision-making.

4. Identifying the Direction of Change in Data

One of the primary objectives of time series forecasting is to understand the direction in which data is heading. Trends indicate whether a variable is experiencing consistent growth, decline, or stability over time. For example, if a company's revenue shows a long-term upward trend despite short-term fluctuations, it suggests business growth. Identifying these trends helps organizations plan future strategies, allocate resources effectively, and respond to market changes efficiently.

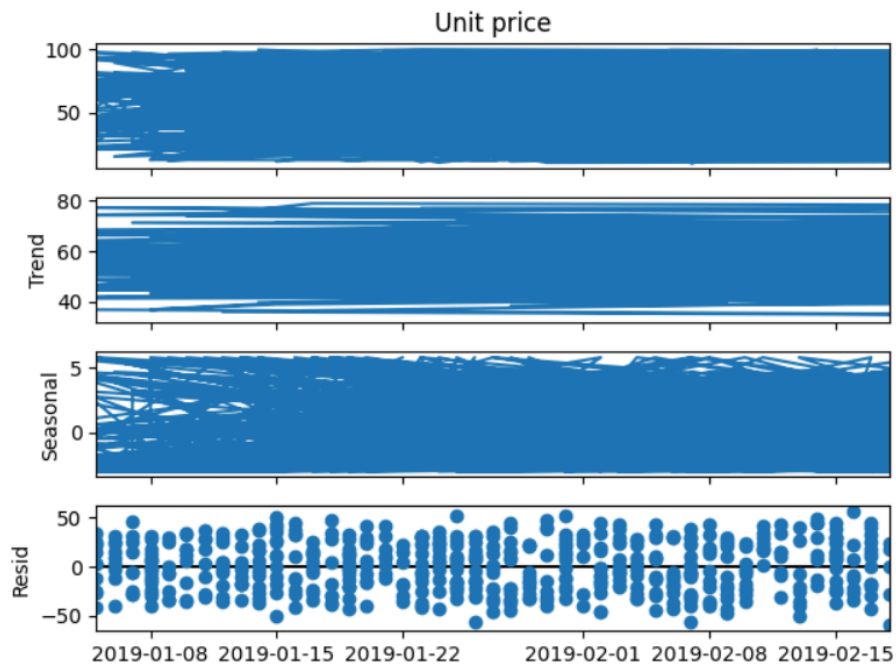
Code & Output:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from statsmodels.tsa.seasonal import seasonal_decompose
from statsmodels.graphics.tsaplots import plot_acf, plot_pacf
from statsmodels.tsa.stattools import adfuller
from statsmodels.tsa.arima.model import ARIMA
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_absolute_error,
mean_squared_error
import warnings
warnings.filterwarnings("ignore")

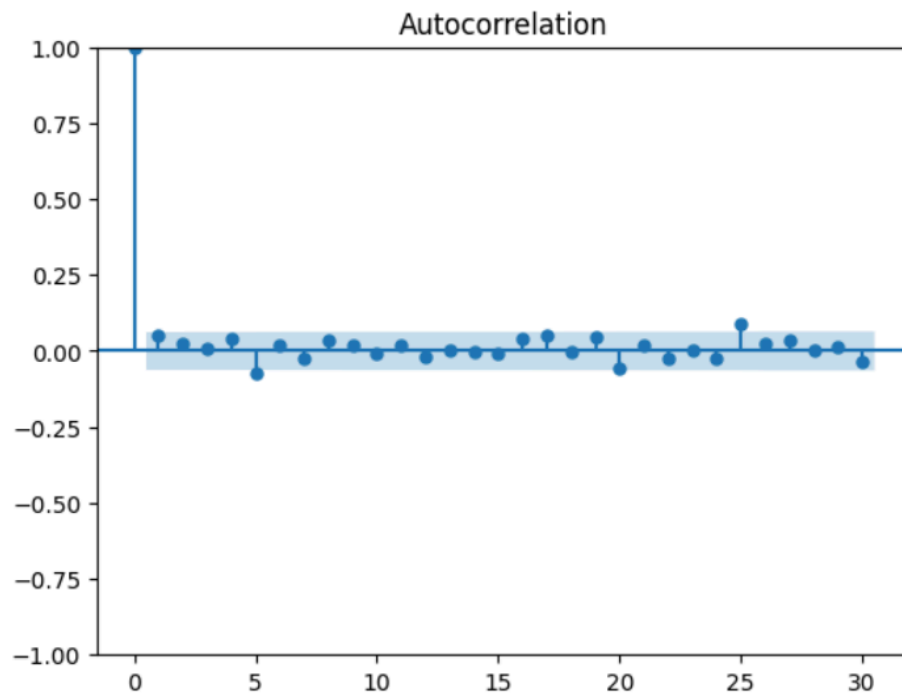
# Load dataset
df = pd.read_csv("supermarket_sales.csv", parse_dates=['Date'])
df.set_index('Date', inplace=True)
print(df.info())

<class 'pandas.core.frame.DataFrame'>
DatetimeIndex: 1000 entries, 2019-01-05 to 2019-02-18
Data columns (total 16 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   Invoice ID                            1000 non-null   object
1   Branch                               1000 non-null   object
2   City                                  1000 non-null   object
3   Customer type                         1000 non-null   object
4   Gender                               1000 non-null   object
5   Product line                         1000 non-null   object
6   Unit price                           1000 non-null   float64
7   Quantity                             1000 non-null   int64
8   Tax 5%                               1000 non-null   float64
9   Total                                1000 non-null   float64
10  Time                                 1000 non-null   object
11  Payment                              1000 non-null   object
12  cogs                                 1000 non-null   float64
13  gross margin percentage              1000 non-null   float64
14  gross income                         1000 non-null   float64
15  Rating                               1000 non-null   float64
dtypes: float64(7), int64(1), object(8)
memory usage: 132.8+ KB
None

# Decompose time series data
result = seasonal_decompose(df['Unit price'], model='additive',
period=12)
result.plot()
plt.show()
```

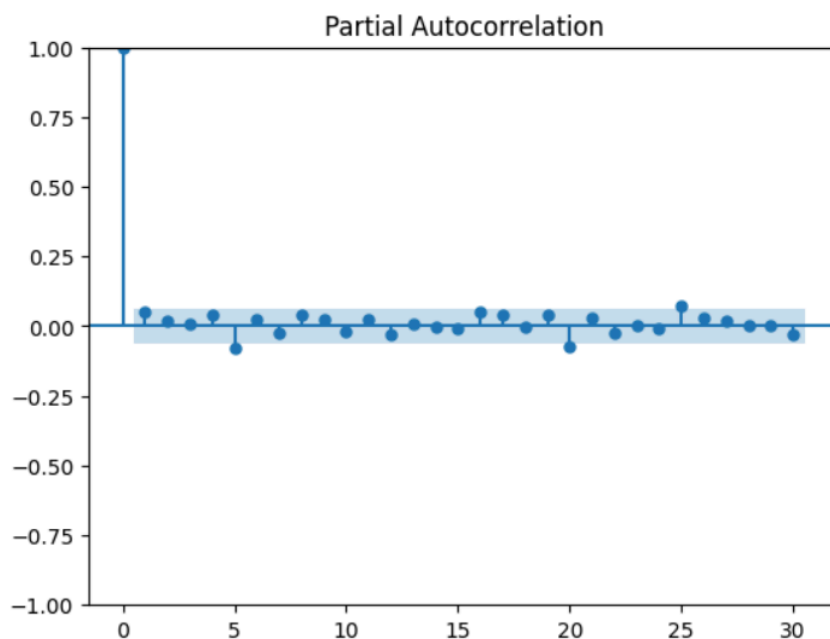


```
# ACF plot
plt.figure(figsize=(12, 5))
plot_acf(df['Quantity'].dropna(), lags=30)
plt.show()
<Figure size 1200x500 with 0 Axes>
```



```
# PACF plot
plt.figure(figsize=(12, 5))
plot_pacf(df['Quantity'].dropna(), lags=30)
plt.show()
```

<Figure size 1200x500 with 0 Axes>



```
# ADF test
adf_result = adfuller(df['Unit price'].dropna())
print(f"ADF Statistic: {adf_result[0]}")
print(f"p-value: {adf_result[1]}")
print("Critical Values:")
for key, value in adf_result[4].items():
    print(f"    {key}: {value}")

ADF Statistic: -30.7870337803048
p-value: 0.0
Critical Values:
    1%: -3.4369127451400474
    5%: -2.864437475834273
    10%: -2.568312754566378

# Time series forecasting using Linear Regression
x = df['Quantity'].values.reshape(-1, 1)
y = df['Total'].values.reshape(-1, 1)

x_train, x_test, y_train, y_test = train_test_split(x, y,
test_size=0.3, random_state=42)

model = LinearRegression()
model.fit(x_train, y_train)
y_pred = model.predict(x_test)
print(y_pred)
```

```

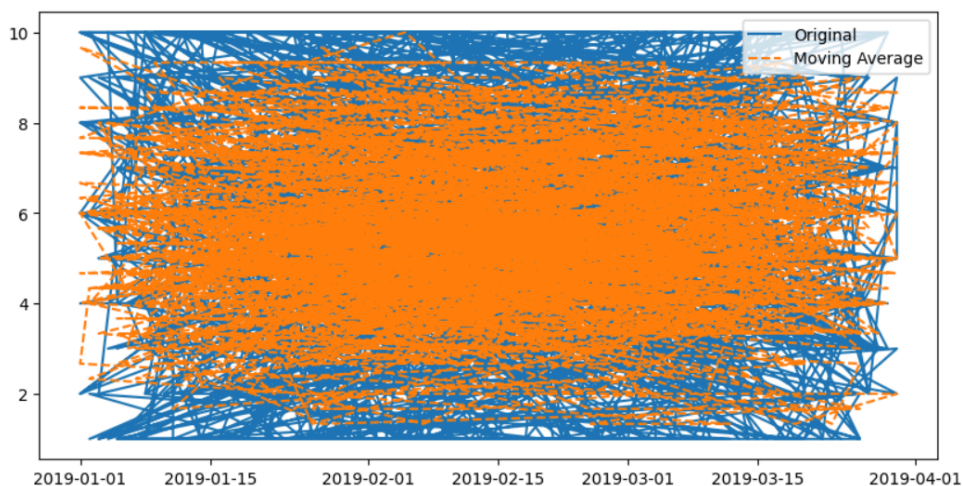
[[346.9113145 ]
 [575.80261581]
 [404.13413983]
 [175.24283853]
 [118.0200132 ]
 [575.80261581]
 [118.0200132 ]
 [404.13413983]
 [518.57979048]
 [289.68848918]
 [575.80261581]
 [518.57979048]
 [404.13413983]
 [175.24283853]
 [175.24283853]
 [518.57979048]
 [461.35696516]
 [346.9113145 ]
 [575.80261581]
 [346.9113145 ]

```

```

# Moving average smoothing model
df['Moving_Avg'] = df['Quantity'].rolling(window=3).mean()
plt.figure(figsize=(10, 5))
plt.plot(df.index, df['Quantity'], label='Original')
plt.plot(df.index, df['Moving_Avg'], label='Moving Average',
linestyle='dashed')
plt.legend()
plt.show()

```



```

# ARIMA model
train_size = int(len(df) * 0.8)
train, test = df.iloc[:train_size], df.iloc[train_size:]

p, d, q = 1, 1, 1
arima_model = ARIMA(train['Unit price'], order=(p, d, q))

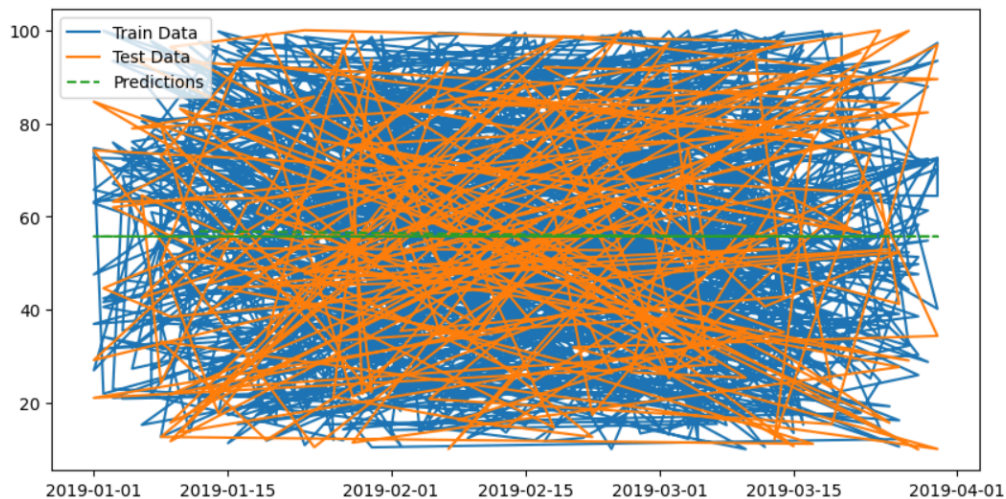
```

```

arima_fit = arima_model.fit()
predictions = arima_fit.forecast(steps=len(test))

plt.figure(figsize=(10, 5))
plt.plot(train.index, train['Unit price'], label='Train Data')
plt.plot(test.index, test['Unit price'], label='Test Data')
plt.plot(test.index, predictions, label='Predictions',
linestyle='dashed')
plt.legend()
plt.show()

```



```

# Model Evaluation
mae = mean_absolute_error(y_test, y_pred)
mape = np.mean(np.abs((y_test - y_pred) / y_test)) * 100
mse = mean_squared_error(y_test, y_pred)
rmse = np.sqrt(mse)

print(f"Mean Absolute Error (MAE): {mae}")
print(f"Mean Absolute Percentage Error (MAPE): {mape}%")
print(f"Mean Squared Error (MSE): {mse}")
print(f"Root Mean Squared Error (RMSE): {rmse}")

Mean Absolute Error (MAE): 133.9579934024882
Mean Absolute Percentage Error (MAPE): 69.91018560406009%
Mean Squared Error (MSE): 31519.30047347451
Root Mean Squared Error (RMSE): 177.53675809103453

```

EXPERIMENT 7

AIM: Outlier detection using distance based/density based method

Theory:

An outlier is a data point that significantly deviates from the rest of the dataset. It does not conform to the general pattern of the data and may result from measurement errors, variability in data, or genuine anomalies. Outliers can distort statistical analyses, affect machine learning models, and lead to incorrect conclusions if not properly handled.

Types of Outliers

1. Global Outliers (Point Anomalies)

- A data point that differs significantly from the entire dataset.
- Example:
 - In a customer database, most users have an account balance between \$1,000 and \$10,000, but one customer has a balance of \$500,000—this is a global outlier.

2. Contextual Outliers (Conditional Anomalies)

- A data point that is normal in one context but abnormal in another.
- Example:
 - A high electricity bill of \$500 in summer for an individual might be normal due to air conditioning use, but the same bill in winter could indicate an anomaly.

3. Collective Outliers (Group Anomalies)

- A group of data points that together deviate from the norm, even though individual values might not be outliers.
- Example:
 - A sudden burst of small transactions within seconds on a bank account (e.g., 50 transfers of \$5) may indicate a fraud attempt, even though individual transactions appear normal.

Why is Outlier Detection Important?

1. Improves Data Quality and Accuracy

- Outliers can skew trends and forecasts in datasets, affecting data interpretation.
- In business analytics, they can lead to poor strategic decisions based on incorrect insights.

2. Impact on Machine Learning Models

- Supervised learning: Outliers can bias model training, leading to incorrect predictions.
- Unsupervised learning: Clustering algorithms like K-Means and DBSCAN may fail to correctly classify data if outliers are present.

3. Fraud Detection & Anomaly Detection

- Outlier detection is widely used in fraud detection for banking, insurance, and cybersecurity.
- Example:
 - A credit card being used in two different countries within minutes may indicate fraud.

4. Industrial & Medical Applications

- In manufacturing, outlier detection helps identify faulty products on an assembly line.
- In healthcare, unusual spikes in patient vital signs could signal a medical emergency.

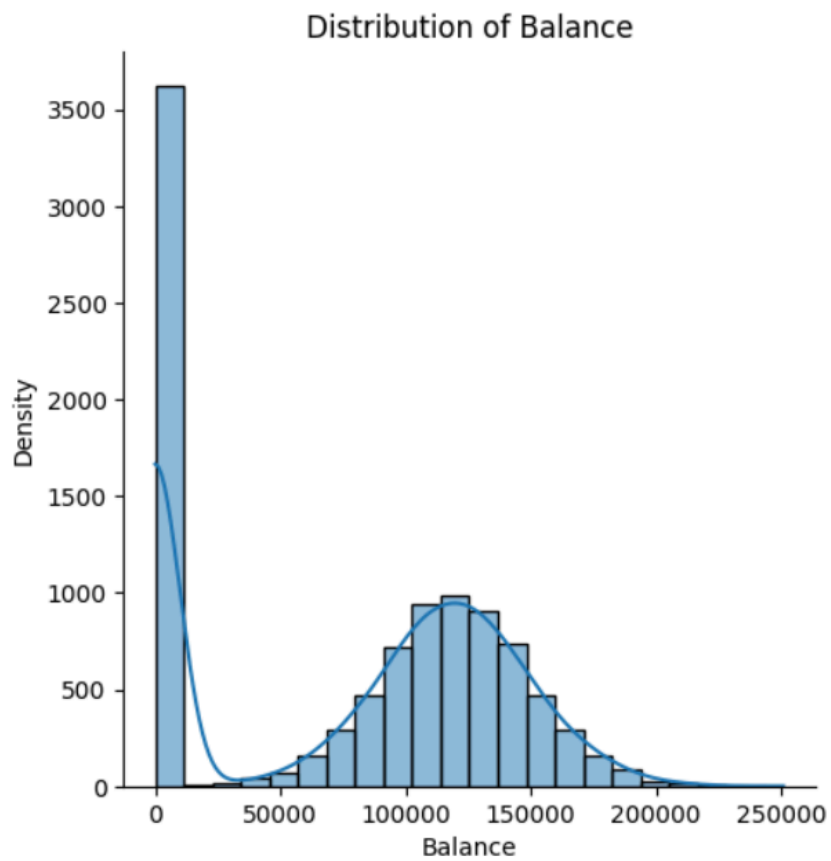
Code & Output:

```
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.neighbors import NearestNeighbors, LocalOutlierFactor
from sklearn.cluster import DBSCAN

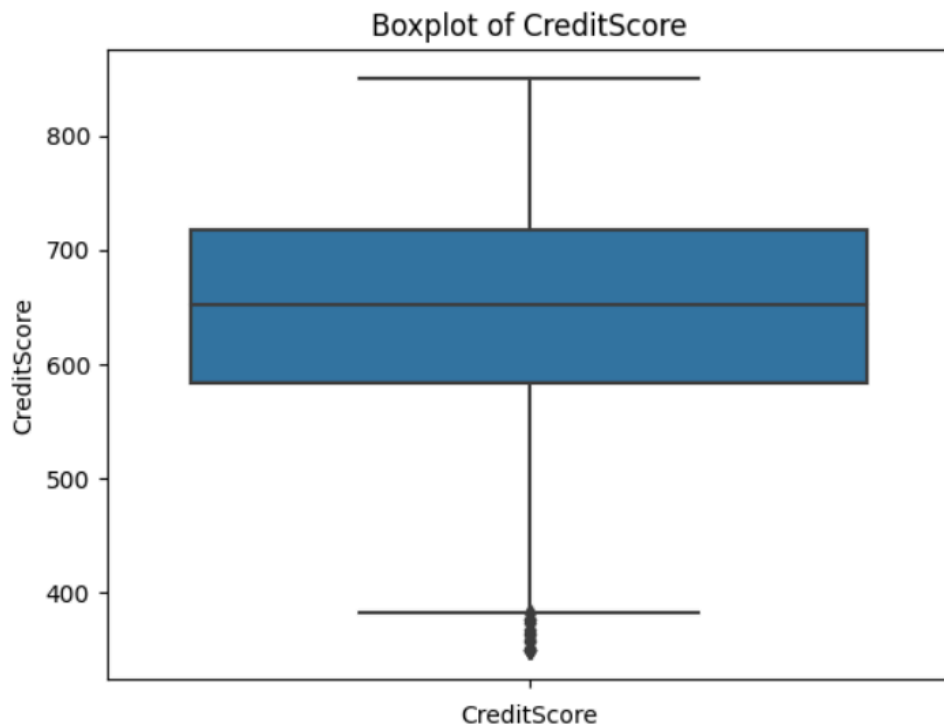
# Load dataset
df = pd.read_csv("Churn_Modelling.csv")
```



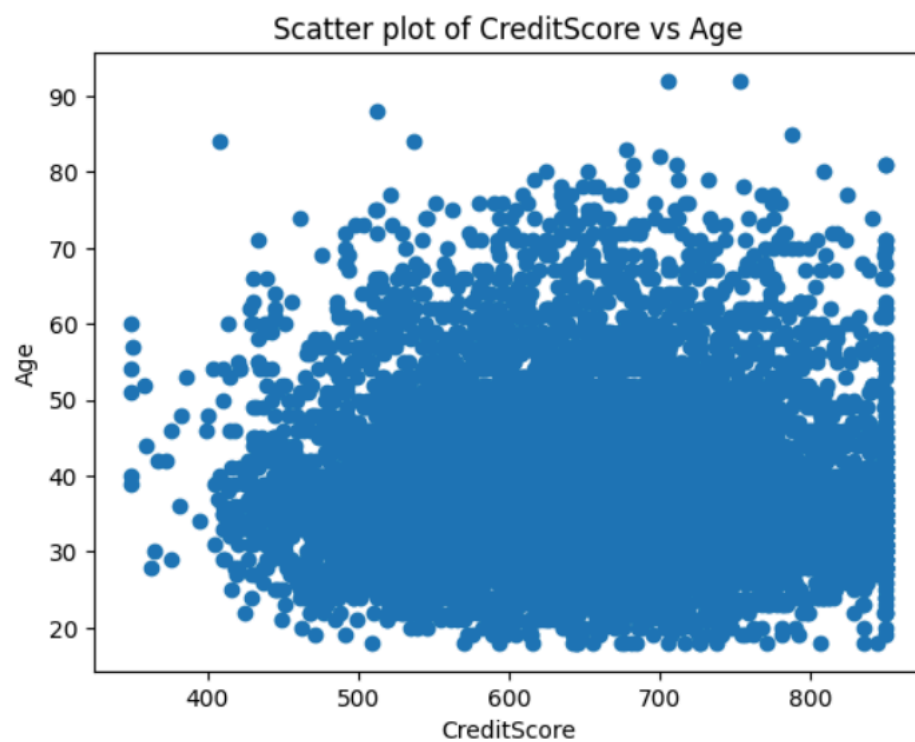
```
# Distribution plot
sns.displot(df['Balance'], kde=True)
plt.xlabel('Balance')
plt.ylabel('Density')
plt.title('Distribution of Balance')
plt.show()
```



```
# Boxplot (Whisker plot)
sns.boxplot(y=df['CreditScore'])
plt.xlabel('CreditScore')
plt.title('Boxplot of CreditScore')
plt.show()
```

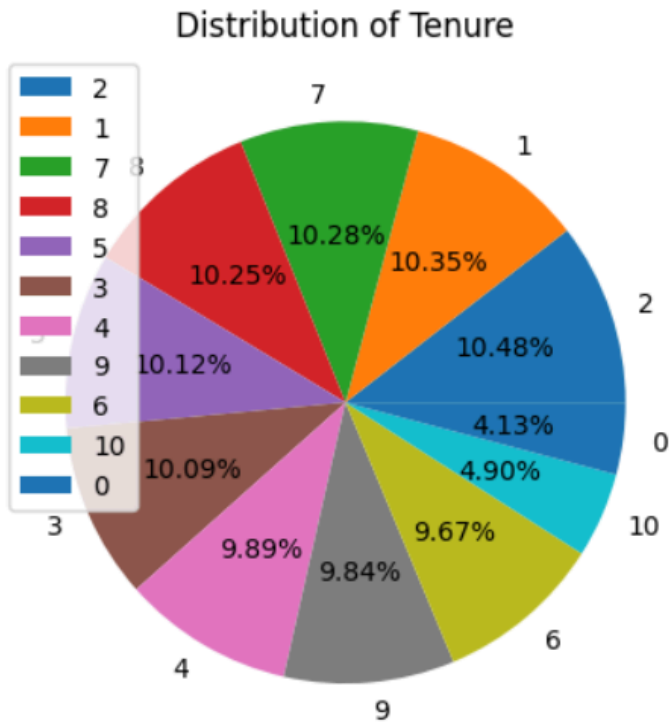


```
# Scatter plot
plt.scatter(df['CreditScore'], df['Age'])
plt.xlabel('CreditScore')
plt.ylabel('Age')
plt.title('Scatter plot of CreditScore vs Age')
plt.show()
```



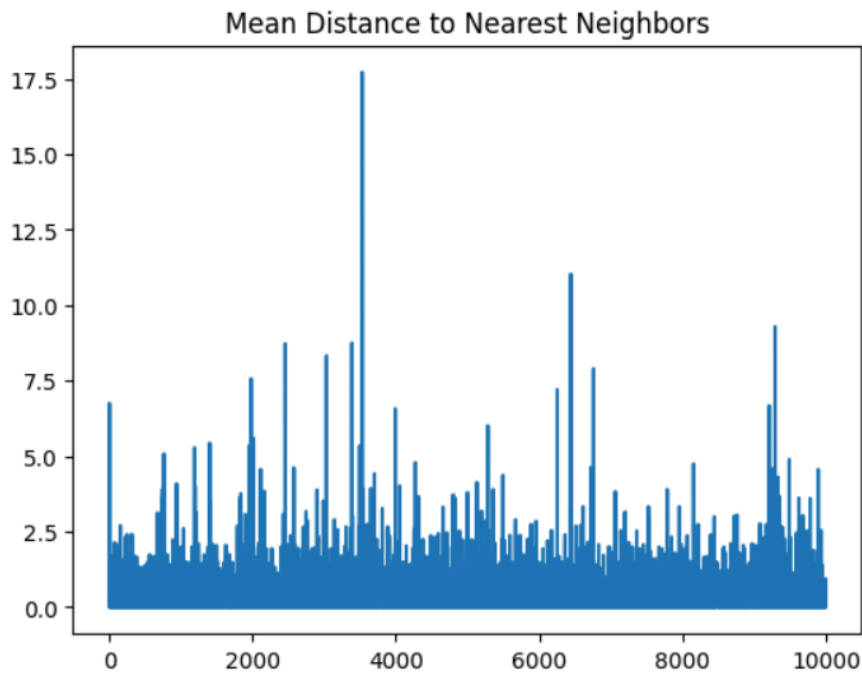
```
# Pie-chart for Tenure
data = df['Tenure'].value_counts().reset_index()
data.columns = ['Tenure', 'Count'] # Renaming for clarity

plt.pie(data['Count'], labels=data['Tenure'], autopct="%1.2f%%")
plt.legend()
plt.title("Distribution of Tenure")
plt.show()
```



```
# Distance-Based Outlier Detection
X = df[['CreditScore', 'Age']].values
nbrs = NearestNeighbors(n_neighbors=3)
nbrs.fit(X)
dist, indexes = nbrs.kneighbors(X)

plt.plot(dist.mean(axis=1))
plt.title("Mean Distance to Nearest Neighbors")
plt.show()
```



```

outlier_index = np.where(dist.mean(axis=1) >
np.percentile(dist.mean(axis=1), 95))
outlier_values = df.iloc[outlier_index]
print("Distance-based Outliers:")
print(outlier_values[['CreditScore', 'Age']])

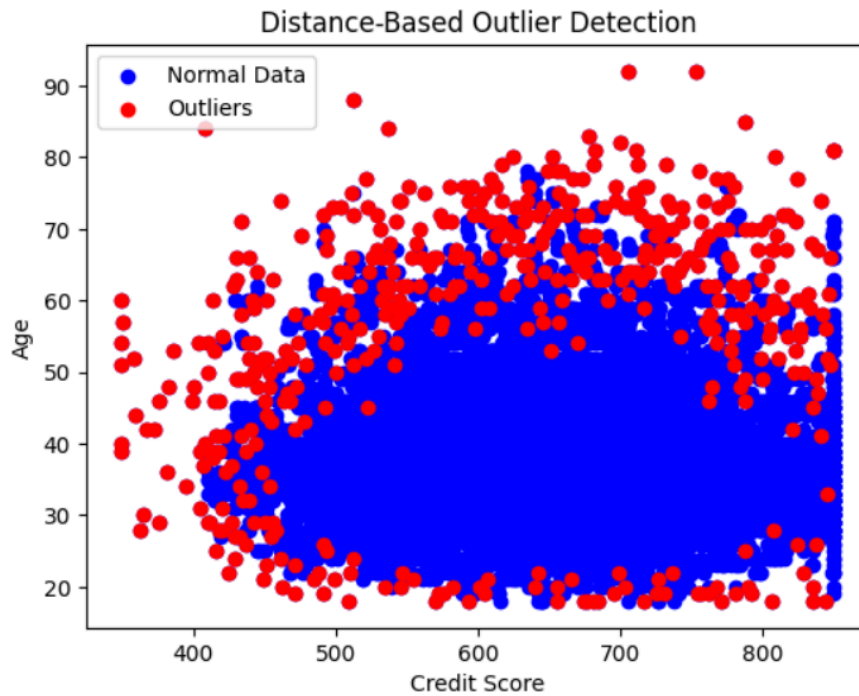
# Plot outliers
plt.scatter(df['CreditScore'], df['Age'], color="b", label='Normal
Data')
plt.scatter(outlier_values['CreditScore'], outlier_values['Age'],
color="r", label='Outliers')
plt.xlabel('Credit Score')
plt.ylabel('Age')
plt.legend()
plt.title("Distance-Based Outlier Detection")
plt.show()

```

Distance-based Outliers:

	CreditScore	Age
6	822	50
7	376	29
29	411	29
57	725	19
58	511	66
...	...	...
9832	595	64
9894	521	77
9908	492	45
9932	677	18
9936	609	77

[480 rows x 2 columns]



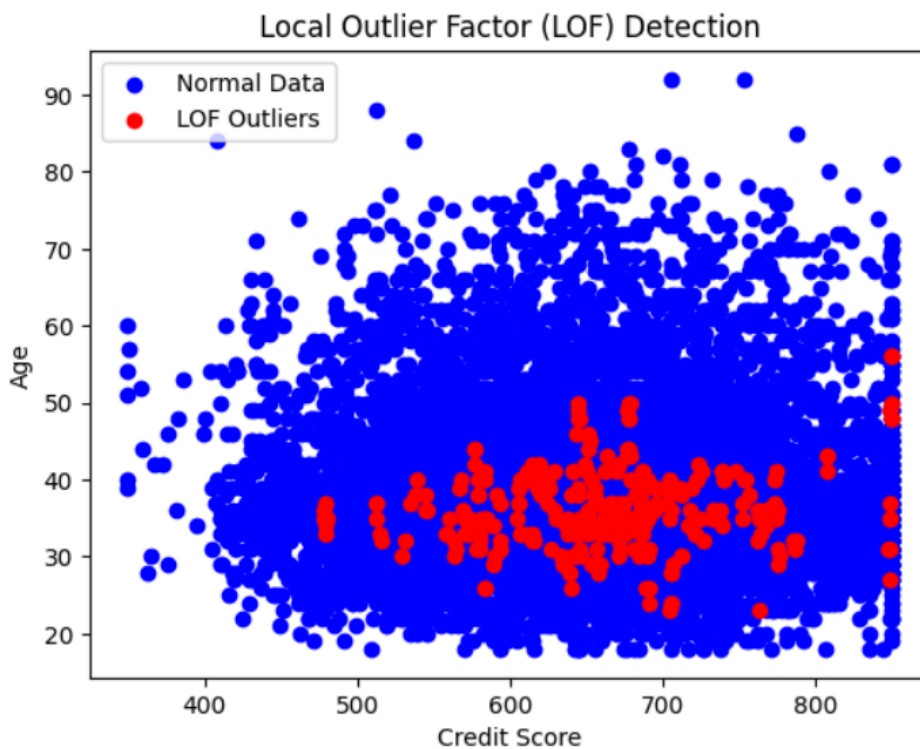
```
# Density-Based Outlier Detection (LOF)
lof = LocalOutlierFactor(n_neighbors=3, contamination=0.05)
preds = lof.fit_predict(X)
outliers = np.where(preds == -1)[0]
outlier_values = df.iloc[outliers]
print("LOF Outliers:")
print(outlier_values[['CreditScore', 'Age']])

# Plot LOF outliers
plt.scatter(df['CreditScore'], df['Age'], color="b", label='Normal
Data')
plt.scatter(outlier_values['CreditScore'], outlier_values['Age'],
color="r", label='LOF Outliers')
plt.xlabel('Credit Score')
plt.ylabel('Age')
plt.legend()
plt.title("Local Outlier Factor (LOF) Detection")
plt.show()
```

LOF Outliers:

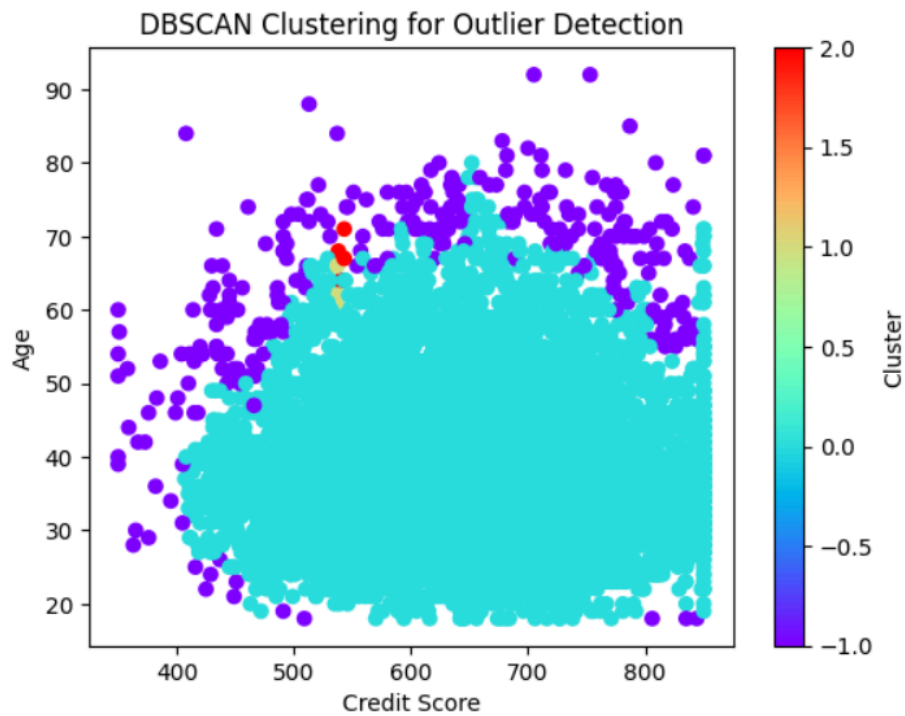
	CreditScore	Age
0	619	42
144	691	31
161	684	30
196	616	32
218	807	43
...	...	...
9867	787	32
9870	607	40
9906	579	38
9946	669	33
9964	479	34

[326 rows x 2 columns]



```
# Density-Based Outlier Detection (DBSCAN)
dbscan = DBSCAN(eps=5, min_samples=10)
df['Cluster'] = dbscan.fit_predict(df[['CreditScore', 'Age']])
colors = df['Cluster']

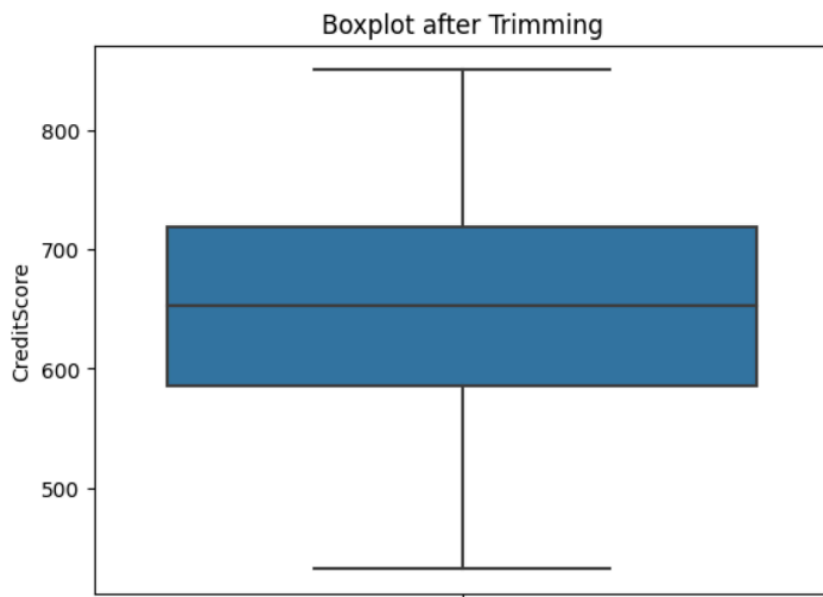
plt.scatter(df['CreditScore'], df['Age'], c=colors,
            cmap='rainbow')
plt.xlabel('Credit Score')
plt.ylabel('Age')
plt.title("DBSCAN Clustering for Outlier Detection")
plt.colorbar(label='Cluster')
plt.show()
```



```
# Removing Outliers (Trimming) upper_limit =
df['CreditScore'].quantile(0.99)
lower_limit = df['CreditScore'].quantile(0.01)

trimmed_df = df[(df['CreditScore'] <= upper_limit) &
(df['CreditScore'] >= lower_limit)]

sns.boxplot(y=trimmed_df['CreditScore'])
plt.title('Boxplot after Trimming')
plt.show()
```

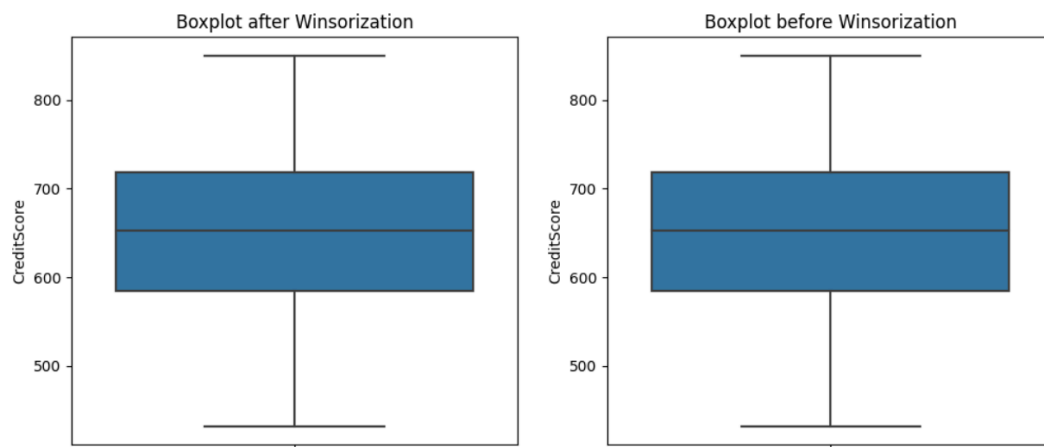


```

# Removing Outliers (Winsorization)
df['CreditScore'] = np.where(df['CreditScore'] >= upper_limit,
                             upper_limit,
                             np.where(df['CreditScore'] <=
lower_limit, lower_limit, df['CreditScore']))

# Boxplot before and after Winsorization
fig, axes = plt.subplots(1, 2, figsize=(12, 5))
sns.boxplot(y=df['CreditScore'], ax=axes[0])
axes[0].set_title('Boxplot after Winsorization')
sns.boxplot(y=df['CreditScore'], ax=axes[1])
axes[1].set_title('Boxplot before Winsorization')
plt.show()

```



EXPERIMENT 8

AIM: Use SMOTE technique to generate synthetic data (to solve the problem of class imbalance)

Theory:

Class imbalance occurs when one class in a dataset has significantly more samples than another. This imbalance can lead to biased models that favor the majority class while poorly predicting the minority class. Several techniques can be used to handle class imbalance:

1. Data-Level Approaches

These methods focus on modifying the dataset before training the model.

a. Oversampling:

- Increases the number of samples in the minority class.
- Example: **SMOTE (Synthetic Minority Over-sampling Technique)** generates synthetic samples instead of duplicating existing ones.

b. Undersampling:

- Reduces the number of samples in the majority class to balance the dataset.
- Example: **Random Undersampling (RUS)** removes some majority class samples.
- Risk: May lose valuable information.

c. Hybrid Methods:

- Combine oversampling and undersampling to achieve better balance.
- Example: **SMOTE + Tomek Links** removes overlapping majority class samples after applying SMOTE.

2. Algorithm-Level Approaches

These methods modify the learning algorithm to be more sensitive to class imbalance.

a. Cost-Sensitive Learning:

- Assigns a higher misclassification cost to the minority class.
- Used in Decision Trees, SVMs, and Neural Networks.

b. Class Weight Adjustment:

- Adjusts class weights in algorithms like Random Forest and Logistic Regression (`class_weight="balanced"` in Scikit-Learn).

c. Anomaly Detection Methods:

- Treats the minority class as an anomaly.
- Example: Isolation Forest, One-Class SVM.

3. Ensemble Methods

These methods combine multiple models to improve minority class prediction.

a. Balanced Bagging & Boosting:

- Randomly samples from both classes in every iteration.
- Example: **Balanced Random Forest, AdaBoost with balanced weights.**

b. EasyEnsemble & BalanceCascade:

- Train multiple classifiers on different balanced subsets.

Working of SMOTE (Synthetic Minority Oversampling Technique)

SMOTE is an oversampling technique that generates synthetic samples for the minority class instead of simply duplicating existing data points.

Steps in SMOTE Algorithm:

1. Select a minority class sample randomly.
2. Find k-nearest neighbors (default k=5).
3. Randomly choose one of these k-nearest neighbors.
4. Generate a new synthetic point along the line connecting the chosen sample and its neighbor.
5. Repeat until the desired class balance is achieved.

Mathematical Formula:

If X_i is the minority class instance and X_{nn} is a randomly chosen neighbor, the synthetic sample X_{new} is created as:

$$X_{new} = X_i + \lambda \times (X_{nn} - X_i)$$

where λ is a random number between 0 and 1.

Code & Output:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import classification_report, f1_score
from imblearn.over_sampling import SMOTE
# Load the churn modeling dataset
data = pd.read_csv('Churn_Modelling.csv')
print("Initial dataset shape:", data.shape)
print("First few rows:")
print(data.head())
```

Initial dataset shape: (10000, 14)
First few rows:

	RowNumber	CustomerId	Surname	CreditScore	Geography	Gender	Age
0	1	15634602	Hargrave	619	France	Female	42 \
1	2	15647311	Hill	608	Spain	Female	41
2	3	15619304	Onio	502	France	Female	42
3	4	15701354	Boni	699	France	Female	39
4	5	15737888	Mitchell	850	Spain	Female	43

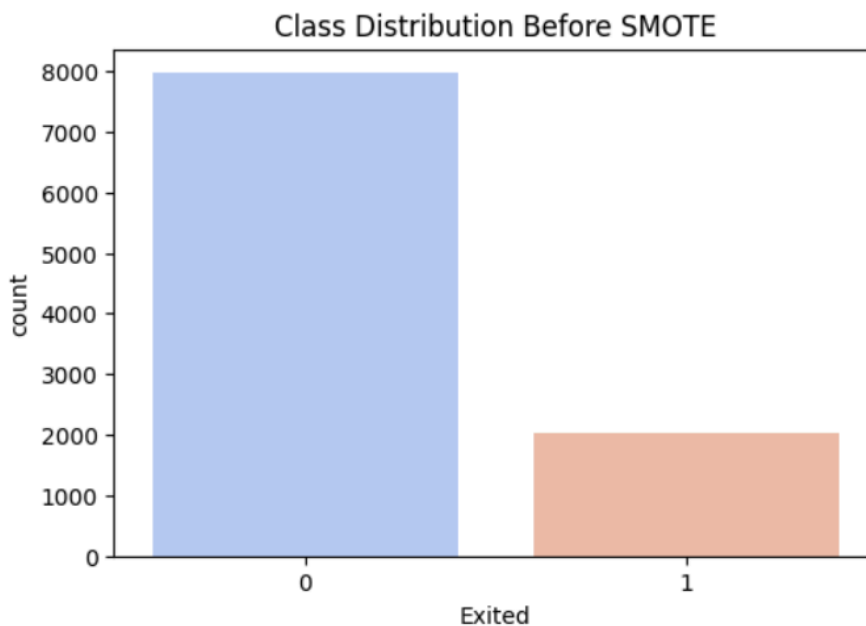
	Tenure	Balance	NumOfProducts	HasCrCard	IsActiveMember
0	2	0.00	1	1	1 \
1	1	83807.86	1	0	1
2	8	159660.80	3	1	0
3	1	0.00	2	0	0
4	2	125510.82	1	1	1

	EstimatedSalary	Exited
0	101348.88	1
1	112542.58	0
2	113931.57	1
3	93826.63	0
4	79084.10	0

```
# Selecting features and target variable
target_column = 'Exited'
features = ['CreditScore', 'Age', 'Tenure', 'Balance',
'NumOfProducts', 'HasCrCard', 'IsActiveMember', 'EstimatedSalary']
X = data[features]
y = data[target_column]
```

```
# 1) Analyze the class label distribution
plt.figure(figsize=(6, 4))
sns.countplot(x=y, palette='coolwarm')
plt.title("Class Distribution Before SMOTE")
plt.show()
print("Class distribution before SMOTE:")
print(y.value_counts())

# Splitting dataset before handling imbalance
X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.2, random_state=42, stratify=y)
```



```
Class distribution before SMOTE:
Exited
0    7963
1     2037
Name: count, dtype: int64
```

```
# 2) Train model without handling imbalance
clf = RandomForestClassifier(random_state=42)
clf.fit(X_train, y_train)
y_pred = clf.predict(X_test)
print("\nEvaluation before applying SMOTE:")
print(classification_report(y_test, y_pred))
print("F1-Score:", f1_score(y_test, y_pred, average='weighted'))
```

Evaluation before applying SMOTE:				
	precision	recall	f1-score	support
0	0.87	0.96	0.91	1593
1	0.75	0.43	0.54	407
accuracy			0.85	2000
macro avg	0.81	0.70	0.73	2000
weighted avg	0.84	0.85	0.84	2000

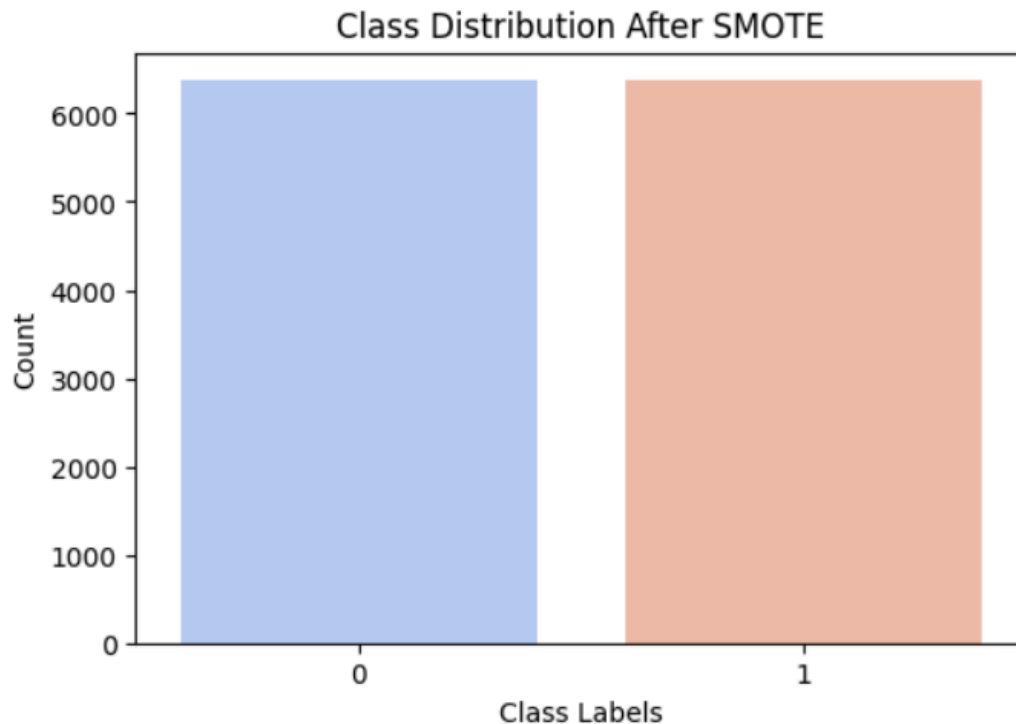
F1-Score: 0.838364226218164

```
# 3) Apply SMOTE to handle class imbalance
# Ensure y_train is properly formatted
y_train = np.array(y_train)

# Apply SMOTE to handle class imbalance
smote = SMOTE(sampling_strategy="auto", k_neighbors=3,
random_state=42)
X_resampled, y_resampled = smote.fit_resample(X_train, y_train)

# Analyzing class distribution after SMOTE
plt.figure(figsize=(6, 4))
sns.barplot(x=np.unique(y_resampled), y=np.bincount(y_resampled),
palette="coolwarm")
plt.title("Class Distribution After SMOTE")
plt.xlabel("Class Labels")
plt.ylabel("Count")
plt.show()

# Print class distribution
print("Class distribution after SMOTE:")
print(np.unique(y_resampled, return_counts=True))
```



Class distribution after SMOTE:

```
(array([0, 1], dtype=int64), array([6370, 6370], dtype=int64))
```

```
# 4) Train model after applying SMOTE
clf_smote = RandomForestClassifier(random_state=42)
clf_smote.fit(X_resampled, y_resampled)
y_pred_smote = clf_smote.predict(X_test)
print("\nEvaluation after applying SMOTE:")
print(classification_report(y_test, y_pred_smote))
print("F1-Score:", f1_score(y_test, y_pred_smote,
average='weighted'))

print("\nComparison before and after SMOTE:")
print("Before SMOTE F1-Score:", f1_score(y_test, y_pred,
average='weighted'))
print("After SMOTE F1-Score:", f1_score(y_test, y_pred_smote,
average='weighted'))
```

Evaluation after applying SMOTE:

	precision	recall	f1-score	support
0	0.89	0.84	0.86	1593
1	0.48	0.60	0.54	407
accuracy			0.79	2000
macro avg	0.69	0.72	0.70	2000
weighted avg	0.81	0.79	0.80	2000

F1-Score: 0.7967107952281

Comparison before and after SMOTE:

Before SMOTE F1-Score: 0.838364226218164

After SMOTE F1-Score: 0.7967107952281

EXPERIMENT 9

AIM: Inferential statistics - Hypothesis testing, Z test, T-test , Anova

Theory:

When working with data, understanding its statistical properties is crucial for making informed decisions. Various inferential statistics techniques help analyze and interpret the data, estimate population parameters, and test hypotheses. Below are key statistical methods and concepts used in data analysis.

1. Inferential Statistics

Inferential statistics allows us to draw conclusions about a population based on a sample. It includes distribution analysis, confidence intervals, and hypothesis testing to make predictions and validate findings.

a) Distributions

A distribution represents how data values are spread over a range. Common probability distributions include:

1. Normal Distribution (Gaussian Distribution):

- Bell-shaped curve, symmetric around the mean.
- Many natural phenomena (e.g., height, weight) follow this distribution.
- Defined by **mean (μ)** and **standard deviation (σ)**.

2. Poisson Distribution:

- Used for modeling rare events (e.g., number of calls at a help desk per hour).
- Defined by a single parameter: **λ (mean occurrence rate per interval)**.

3. Population Parameters vs. Sample Statistics:

- **Population parameters** (e.g., population mean μ , population variance σ^2) describe the entire dataset.
- **Sample statistics** (e.g., sample mean $\bar{X}$, sample variance S^2) estimate population parameters based on a sample.

4. Sampling Errors:

- The difference between a sample statistic and the true population parameter.
- Caused by variability in the sample selection process.

b) Confidence Intervals (CI)

A confidence interval provides a range within which we expect the true population parameter to lie with a certain probability (e.g., 95%).

Formula for Confidence Interval for the Mean:

$$CI = \bar{X} \pm Z \times \left(\frac{\sigma}{\sqrt{n}} \right)$$

where:

- $\bar{X}$ = Sample mean
- Z = Z-score (based on confidence level, e.g., 1.96 for 95%)
- σ = Standard deviation
- n = Sample size

Example: A 95% confidence interval of (50, 60) means we are 95% confident that the population mean lies between 50 and 60.

c) Hypothesis Testing

Hypothesis testing is a statistical method used to make decisions about a population parameter based on sample data.

Steps in Hypothesis Testing:

1. Define **Null Hypothesis (H_0)** and **Alternative Hypothesis (H_1)**.
2. Choose a significance level (α), usually 0.05 (5%).
3. Select the appropriate statistical test (Z-test, t-test, ANOVA, etc.).
4. Calculate the test statistic and p-value.
5. Compare the p-value with α :
 - If $p \leq \alpha$, reject H_0 (significant result).
 - If $p > \alpha$, fail to reject H_0 (not significant).

1) Z-Test

- Used when sample size (n) > 30 and population standard deviation is known.
- Tests whether the sample mean significantly differs from the population mean.

Formula for Z-test:

$$Z = \frac{\bar{X} - \mu}{\frac{\sigma}{\sqrt{n}}}$$

where:

- $\bar{X}$ = Sample mean
- μ = Population mean
- σ = Population standard deviation
- n = Sample size

Example: Checking if the average height of students differs from 170 cm.

2) T-Test

- Used when sample size (n) < 30 or population standard deviation is unknown.
- Compare the means of one or two samples.

Types of t-tests:

1. **One-sample t-test:** Compares sample mean to a known population mean.
2. **Independent (two-sample) t-test:** Compares means of two independent groups.
3. **Paired t-test:** Compares means of the same group before and after a treatment.

Formula for t-test:

$$t = \frac{\bar{X} - \mu}{\frac{S}{\sqrt{n}}}$$

where:

- S = Sample standard deviation

Example: Comparing the effectiveness of two different drugs on blood pressure reduction.

3) ANOVA (Analysis of Variance)

- Used to compare means of three or more groups.
- Determines if at least one group mean is significantly different from the others.

Types of ANOVA:

1. **One-Way ANOVA:** Compares means across one factor (e.g., test scores of students in three different schools).
2. **Two-Way ANOVA:** Compares means across two factors (e.g., test scores based on both school and gender).

Formula for ANOVA (F-statistic):

$$F = \frac{\text{Variance between groups}}{\text{Variance within groups}}$$

- A **higher F-value** suggests significant differences between groups.

Example: Testing if different fertilizers affect plant growth differently.

Code & Output:

```
import numpy as np
import pandas as pd
import scipy.stats as stats
import matplotlib.pyplot as plt
sales_df = pd.read_csv('supermarket_sales.csv')
print("Initial dataset shape:", sales_df.shape)
print("First few rows of the dataset:\n", sales_df.head())
```

Initial dataset shape: (1000, 17)

First few rows of the dataset:

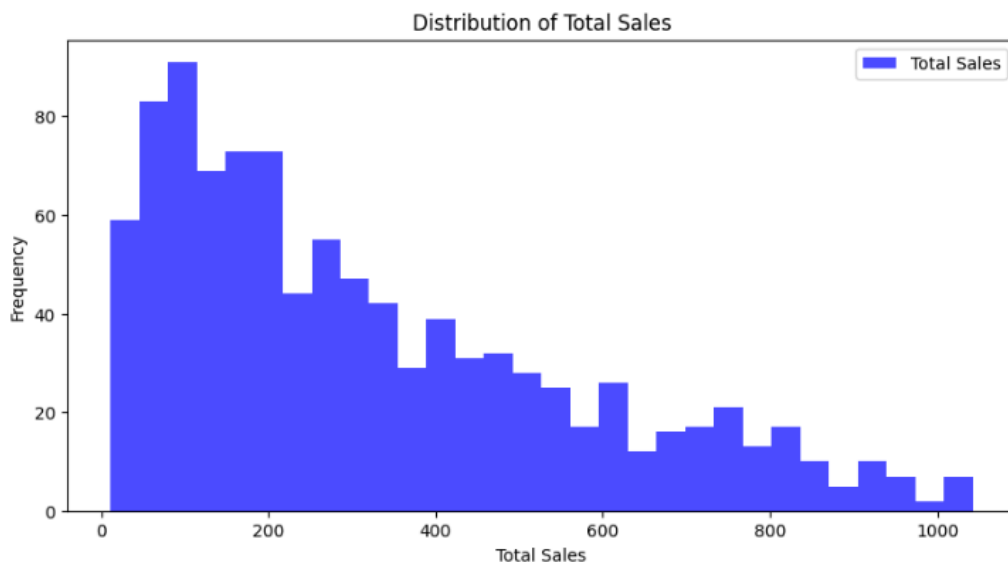
	Invoice ID	Branch	City	Customer type	Gender
0	750-67-8428	A	Yangon	Member	Female \
1	226-31-3081	C	Naypyitaw	Normal	Female
2	631-41-3108	A	Yangon	Normal	Male
3	123-19-1176	A	Yangon	Member	Male
4	373-73-7910	A	Yangon	Normal	Male

	Product line	Unit price	Quantity	Tax 5%	Total	Date
0	Health and beauty	74.69	7	26.1415	548.9715	1/5/2019 \
1	Electronic accessories	15.28	5	3.8200	80.2200	3/8/2019
2	Home and lifestyle	46.33	7	16.2155	340.5255	3/3/2019
3	Health and beauty	58.22	8	23.2880	489.0480	1/27/2019
4	Sports and travel	86.31	7	30.2085	634.3785	2/8/2019

	Time	Payment	cogs	gross margin percentage	gross income	Rating
0	13:08	Ewallet	522.83	4.761905	26.1415	9.1
1	10:29	Cash	76.40	4.761905	3.8200	9.6
2	13:23	Credit card	324.31	4.761905	16.2155	7.4
3	20:33	Ewallet	465.76	4.761905	23.2880	8.4
4	10:37	Ewallet	604.17	4.761905	30.2085	5.3

a) Distributions - Checking Normal and Poisson Distributions

```
plt.figure(figsize=(10, 5))
plt.hist(sales_df['Total'], bins=30, alpha=0.7, color='blue',
label='Total Sales')
plt.title("Distribution of Total Sales")
plt.xlabel("Total Sales")
plt.ylabel("Frequency")
plt.legend()
plt.show()
```



Checking Population Parameters and Sampling Errors

```
sample = sales_df['Total'].sample(50, random_state=42)
print("Sample Mean:", sample.mean())
print("Population Mean:", sales_df['Total'].mean())
print("Sampling Error:", abs(sample.mean() -
sales_df['Total'].mean()))
```

Sample Mean: 308.12103
Population Mean: 322.966749
Sampling Error: 14.845718999999974

```
# b) Confidence Intervals
confidence = 0.95
mean_total = np.mean(sample)
stdev_total = np.std(sample, ddof=1)
n = len(sample)
margin_of_error = stats.t.ppf((1 + confidence) / 2, n - 1) *
(stdev_total / np.sqrt(n))
print("\nConfidence Interval for Total Sales:")
print(f"{confidence*100}% CI: ({mean_total - margin_of_error},
{mean_total + margin_of_error})")
```

Confidence Interval for Total Sales:
95.0% CI: (245.04737425317424, 371.1946857468258)

```
# c) Hypothesis Testing
```

```
# Z-Test: Comparing mean 'Total' sales between Male and Female
customers
print("\nZ-Test (Male vs Female Total Sales):")
print("Null Hypothesis (H0): There is no significant difference in
total sales between Male and Female customers.")
print("Alternative Hypothesis (H1): There is a significant
difference in total sales between Male and Female customers.")
```

```
male_total = sales_df[sales_df['Gender'] == 'Male']['Total']
female_total = sales_df[sales_df['Gender'] == 'Female']['Total']
```

```
z_stat, z_p_value = stats.ttest_ind(male_total, female_total)
print("Z-Statistic:", z_stat)
print("P-Value:", z_p_value)
print("Conclusion:", "Reject Null Hypothesis" if z_p_value < 0.05
else "Fail to Reject Null Hypothesis")
```

```
Z-Test (Male vs Female Total Sales):
Null Hypothesis (H0): There is no significant difference in total sales between Male and Female customers.
Alternative Hypothesis (H1): There is a significant difference in total sales between Male and Female customers.
Z-Statistic: -1.5641266415241026
P-Value: 0.11810484577135677
Conclusion: Fail to Reject Null Hypothesis
```

```
# T-Test: Comparing average 'Unit price' between Member and Normal
customers
print("\nT-Test (Member vs Normal Customer - Unit Price):")
print("Null Hypothesis (H0): There is no significant difference in
unit price between Member and Normal customers.")
print("Alternative Hypothesis (H1): There is a significant
difference in unit price between Member and Normal customers.")
```

```

member_price = sales_df[sales_df['Customer type'] ==
'Member']['Unit price']
normal_price = sales_df[sales_df['Customer type'] ==
'Normal']['Unit price']

t_stat, t_p_value = stats.ttest_ind(member_price, normal_price)
print("T-Statistic:", t_stat)
print("P-Value:", t_p_value)
print("Conclusion:", "Reject Null Hypothesis" if t_p_value < 0.05
else "Fail to Reject Null Hypothesis")

T-Test (Member vs Normal Customer - Unit Price):
Null Hypothesis (H0): There is no significant difference in unit price between Member and Normal customers.
Alternative Hypothesis (H1): There is a significant difference in unit price between Member and Normal customers.
T-Statistic: 0.6394684690480077
P-Value: 0.5226649706896371
Conclusion: Fail to Reject Null Hypothesis

# ANOVA: Comparing 'Rating' across Branches A, B, and C
print("\nANOVA (Comparing Rating across Branches A, B, and C):")
print("Null Hypothesis (H0): There is no significant difference in
customer ratings across different branches.")
print("Alternative Hypothesis (H1): There is a significant
difference in customer ratings across different branches.")

branch_a = sales_df[sales_df['Branch'] == 'A']['Rating']
branch_b = sales_df[sales_df['Branch'] == 'B']['Rating']
branch_c = sales_df[sales_df['Branch'] == 'C']['Rating']

anova_stat, anova_p_value = stats.f_oneway(branch_a, branch_b,
branch_c)
print("ANOVA Statistic:", anova_stat)
print("P-Value:", anova_p_value)
print("Conclusion:", "Reject Null Hypothesis" if anova_p_value <
0.05 else "Fail to Reject Null Hypothesis")

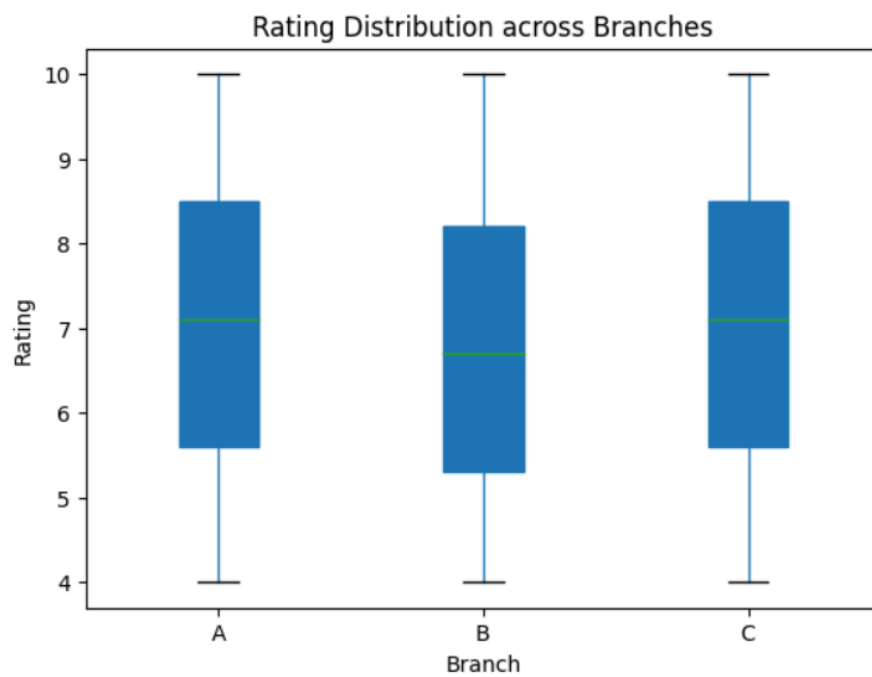
ANOVA (Comparing Rating across Branches A, B, and C):
Null Hypothesis (H0): There is no significant difference in customer ratings across different branches.
Alternative Hypothesis (H1): There is a significant difference in customer ratings across different branches.
ANOVA Statistic: 2.0754771216823635
P-Value: 0.12603841851334457
Conclusion: Fail to Reject Null Hypothesis

# Boxplot for Rating Distribution across Branches
plt.figure(figsize=(8, 5))
sales_df.boxplot(column='Rating', by='Branch', grid=False,
patch_artist=True)
plt.title("Rating Distribution across Branches")
plt.suptitle("")
plt.xlabel("Branch")
plt.ylabel("Rating")
plt.show()

```

```
print("Hypothesis Testing Completed Successfully!")
```

<Figure size 800x500 with 0 Axes>



Hypothesis Testing Completed Successfully!

EXPERIMENT 10

AIM: Demonstration of selected case study - House Price Prediction

Theory:

The real estate market plays a vital role in the global economy, influencing investment decisions, financial stability and individual wealth. Accurate prediction of real estate prices is crucial for both buyers and sellers to make informed decisions. For most people, purchasing a home is a lifelong dream, but many people make missteps when doing so.

Buying overpriced estates that aren't worth is a classical error.

Property prices are influenced by a multitude of intricate factors, ranging from location and property size to economic trends and market demands. The ability to accurately predict real estate prices holds immense value for potential buyers, sellers, investors and agents alike.

This case study aims to utilize the power of linear regression, a fundamental machine learning algorithm, to predict real estate prices based on relevant features. By doing so, we seek to contribute to more informed property transactions and investment strategies.

The Benefits of Using a Real Estate Price Prediction Website:

- **Increased market efficiency:** Real estate price predictions can help to make the real estate market more efficient by providing buyers and sellers with more information. This can lead to faster and more informed transactions.
- **Reduced risk:** Real estate price predictions can help to reduce the risk involved in buying and selling real estate. This can make the real estate market more accessible to a wider range of people.
- **Improved financial well-being:** Real estate is often one of the largest assets that people own. Accurate real estate price predictions can help people to make better financial decisions and improve their overall financial well-being.

Code & Output:

House Price Prediction

ADS EXPERIMENT 10

GROUP 44 | C-33

GROUP MEMBERS

2103021	◆—◆	CHETANA BHOJWANI
2103109	◆—◆	ARNAV MALVIA
2103166	◆—◆	MAITHILI SHINDE



INTRODUCTION



- This project focuses on real estate data analytics and uses data science to predict housing prices accurately.
- The project follows a comprehensive workflow from data cleaning and preprocessing to model training and deployment.
- The approach turns raw data into actionable insights for better real estate decision-making.
- The project automates the prediction process while maintaining high levels of accuracy.



GOALS



- The project aims to provide accurate house price predictions to support strategic investment decisions.
- It emphasizes the practical application of data science techniques, including data preprocessing, statistical analysis, and machine learning.
- The work demonstrates how applied data science can solve real-world challenges in the real estate market.
- The solution is designed to be a scalable framework for continuous improvement and further data exploration.

WORKING

- **Data Handling & Visualization:** The project utilizes Pandas and NumPy for data loading and manipulation, and Matplotlib for creating visualizations such as scatter plots and histograms to explore data distributions and relationships.
- **Data Preprocessing & Feature Engineering:** It involves cleaning the data by removing null values and irrelevant features, standardizing location names, converting square footage ranges to numeric values, and extracting new features like the number of bedrooms (BHK) from the property size information.
- **Outlier Detection & Removal:** The approach includes filtering out properties with unrealistic square footage per bedroom and applying statistical techniques to remove outliers in the price per square feet across different locations.

WORKING

- **Modeling Techniques:** The project implements multiple machine learning models including Linear Regression, Lasso, and Decision Tree Regressor.
- **Model Evaluation:** It employs train-test splits, cross-validation, and hyperparameter tuning using GridSearchCV to ensure the reliability and performance of the models.
- **Prediction Function:** A custom prediction function is developed to estimate house prices based on input features such as location, square footage, number of bathrooms, and BHK count.

IMPLEMENTATION

```
[1] 1 import pandas as pd
    2 import numpy as np
    3 import matplotlib
    4 import matplotlib.pyplot as plt
    5 %matplotlib inline
    6 matplotlib.rcParams['figure.figsize']=(20,10)
```

```
00 1 df1 = pd.read_csv("bengaluru_house_prices.csv")
    2 df1.head()
```

	area_type	availability	location	size	society	total_sqft	bath	balcony	price
0	Super built-up Area	19-Dec	Electronic City Phase II	2 BHK	Comee	1056	2.0	1.0	39.07
1	Plot Area	Ready To Move	Chikka Tirupathi	4 Bedroom	Theanmp	2600	5.0	3.0	120.00
2	Built-up Area	Ready To Move	Uttarahalli	3 BHK	NaN	1440	2.0	3.0	62.00
3	Super built-up Area	Ready To Move	Lingadheeranahalli	3 BHK	Solewre	1521	3.0	1.0	95.00
4	Super built-up Area	Ready To Move	Kothanur	2 BHK	NaN	1200	2.0	1.0	51.00

Next steps: [Generate code with df1](#) [View recommended plots](#) [New interactive sheet](#)

```
00 [3] 1 df1.shape
```

(13320, 9)

IMPLEMENTATION

```
00 [16] 1 def convert_sqft_to_num(x):
    2     tokens = x.split('-')
    3     if len(tokens) == 2:
    4         return (float(tokens[0])+float(tokens[1]))/2
    5     try:
    6         return float(x)
    7     except:
    8         return None
```

```
00 [17] 1 convert_sqft_to_num("3000-2-")
```

```
00 [18] 1 convert_sqft_to_num("4125Perch")
```

```
00 1 df4 = df3.copy()
    2 df4['total_sqft'] = df4['total_sqft'].apply(convert_sqft_to_num)
    3 df4.loc[30]
```

	30
location	Yelahanka
size	4 BHK
total_sqft	2475.0
bath	4.0
price	186.0

IMPLEMENTATION

```
[21] 1 df5['Price_per_sqft'] = df5['price'] * 100000 / df5['total_sqft']
      2 df5.head()
```

	location	size	total_sqft	bath	price	bhk	Price_per_sqft
0	Electronic City Phase II	2 BHK	1056.0	2.0	39.07	2	3699.810606
1	Chikka Tirupathi	4 Bedroom	2600.0	5.0	120.00	4	4615.384615
2	Uttarahalli	3 BHK	1440.0	2.0	62.00	3	4305.555556
3	Lingadheeranahalli	3 BHK	1521.0	3.0	95.00	3	6245.890861
4	Kothanur	2 BHK	1200.0	2.0	51.00	2	4250.000000

Next steps: [Generate code with df5](#) [View recommended plots](#) [New interactive sheet](#)

```
[22] 1 len(df5.location.unique())
```

1304

```
1 df5.location = df5.location.apply(lambda x: x.strip())
2 location_stats = df5.groupby('location')['location'].agg('count').sort_values(ascending=False)
3 location_stats.head(30)
```

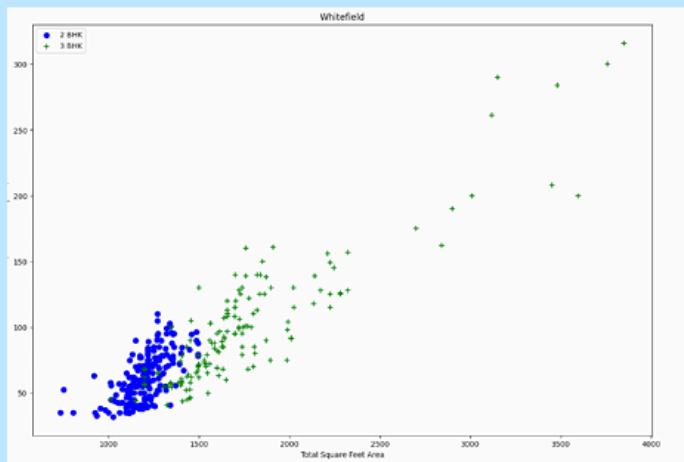
IMPLEMENTATION

```
[31] 1 def remove_pps_outliers(df):
      2     df_out = pd.DataFrame()
      3     for key, subdf in df.groupby('location'):
      4         m = np.mean(subdf.Price_per_sqft)
      5         st = np.std(subdf.Price_per_sqft)
      6         reduced_df = subdf[(subdf.Price_per_sqft > (m - st)) & (subdf.Price_per_sqft <= (m + st))]
      7         df_out = pd.concat([df_out, reduced_df], ignore_index=True)
      8     return df_out
      9 df7 = remove_pps_outliers(df6)
     10 df7.shape
```

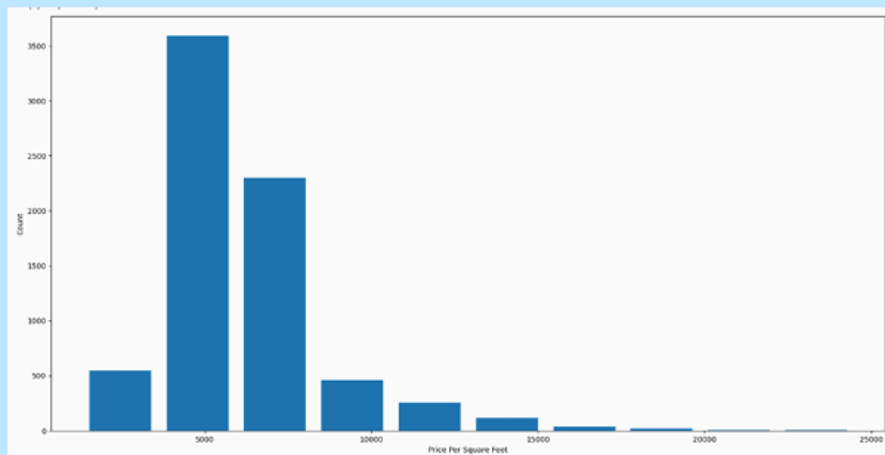
(10241, 7)

```
1 def plot_scatter_chart(df, location):
2     bhk2 = df[(df.location == location) & (df.bhk == 2)]
3     bhk3 = df[(df.location == location) & (df.bhk == 3)]
4     matplotlib.rcParams['figure.figsize'] = (15, 10)
5     plt.scatter(bhk2.total_sqft, bhk2.price, color='blue', label='2 BHK', s=50)
6     plt.scatter(bhk3.total_sqft, bhk3.price, marker='+', color='green', label='3 BHK', s=50)
7     plt.xlabel("Total Square Feet Area")
8     plt.ylabel("Price (Lakh Indian Rupees)")
9     plt.title(location)
10    plt.legend()
11
12 plot_scatter_chart(df7, "Whitefield")
```

IMPLEMENTATION



IMPLEMENTATION



IMPLEMENTATION

```
[48] 1 from sklearn.model_selection import train_test_split
      2 X_train , X_test , y_train, y_test = train_test_split(X,y,test_size=0.2,random_state=10)

[49] 1 from sklearn.linear_model import LinearRegression
      2 model = LinearRegression()
      3 model.fit(X_train,y_train)
      4 model.score(X_test,y_test)

0.8452277697874329

1 from sklearn.model_selection import ShuffleSplit
2 from sklearn.model_selection import cross_val_score
3
4 cv = ShuffleSplit(n_splits=7, test_size=0.2, random_state=0)
5
6 cross_val_score(LinearRegression(), X, y, cv=cv)

array([0.82430186, 0.77166234, 0.85089567, 0.80837764, 0.83653286,
       0.77463657, 0.84724578])
```

IMPLEMENTATION

```
[52] 1 def predict_price(location,sqft,bath,bhk):
      2     loc_index = np.where(X.columns==location)[0][0]
      3
      4     x = np.zeros(len(X.columns))
      5     x[0] = sqft
      6     x[1] = bath
      7     x[2] = bhk
      8     if loc_index >= 0:
      9         x[loc_index] = 1
      10
      11     return model.predict([x])[0]

[57] 1 print(predict_price('Whitefield',2500, 4, 2))

182.63949887342164
/usr/local/lib/python3.11/dist-packages/sklearn/utils/validation.py:100: UserWarning:
  warnings.warn(

1 print(predict_price('Whitefield',1500, 3, 2))

98.14510000448142
/usr/local/lib/python3.11/dist-packages/sklearn/utils/validation.py:100: UserWarning:
  warnings.warn(

[59] 1 print(predict_price('Indira Nagar',1000, 2, 2))

181.27815484006715
```

CONCLUSION

- The project successfully demonstrates the use of data science to predict housing prices, yielding actionable insights and improved decision-making in real estate.
- Future work could include integrating additional features such as neighborhood trends and economic indicators, as well as exploring more advanced machine learning and deep learning models.
- The current system provides a scalable and adaptable framework that can be continuously enhanced to further improve prediction accuracy and real-time responsiveness.

64
109

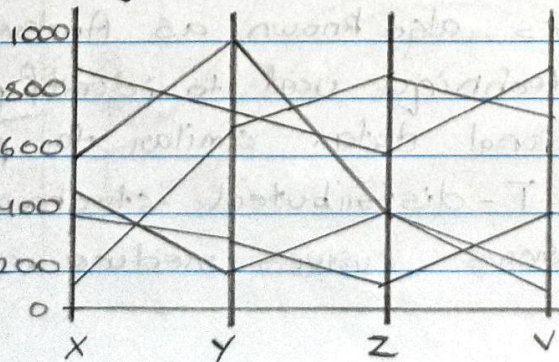
Assignment :- 1

Q. Assignment on visualising high dimensional data:- Parallel chart, Deviation chart, Andrews Curves.

→ When the data has high dimensions, there are patterns hidden in the data that cannot be easily identified by visual observations. This is the main reason visualisation of high dimensional data is important. To achieve this goal of visualisation, dimensionality Reduction is required.

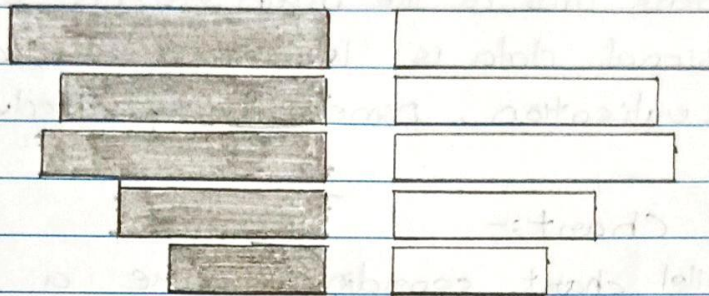
Parallel Chart:-

- 1) A parallel chart coordinates are a common way of visualising and analysing high dimensional datasets.
- 2) To show a set of points in an n -dimensional space, a backdrop is drawn consisting of n parallel lines, typically vertical and equally spaced. A point in n -dimensional space is represented as a polyline with vertices on the parallel axes, the position of the vertex on the i th axis corresponds to the i th coordinate of the point.
- 3) This visualisation is closely related to the time series visualisation, except that it is applied to data where the axes do not correspond to points in time, therefore do not have a natural order. Therefore, different axis arrangements may be of interest.



Deviation Chart:-

- 1) Deviation bar graphs are simply two bar charts aligned, where one of the charts runs right to left rather than left to right. The two charts overlap ~~the~~ on the same categories but differ in terms of respondent group or some other variable.



- 2) Thus the shared categories, such as the survey questions are listed in the space in ^{the} middle of the two bar charts. Usually, these are constructed as bar charts, where the bars are horizontal, not column charts.
- 3) The purpose of graphing each set of responses separately from one another so that it is easier to see the shape of each response set. Thus it is important to order one of the response sets from greatest to least so that it is easier for a viewer to see where the other response set is out of shape.

Andrews Curves :-

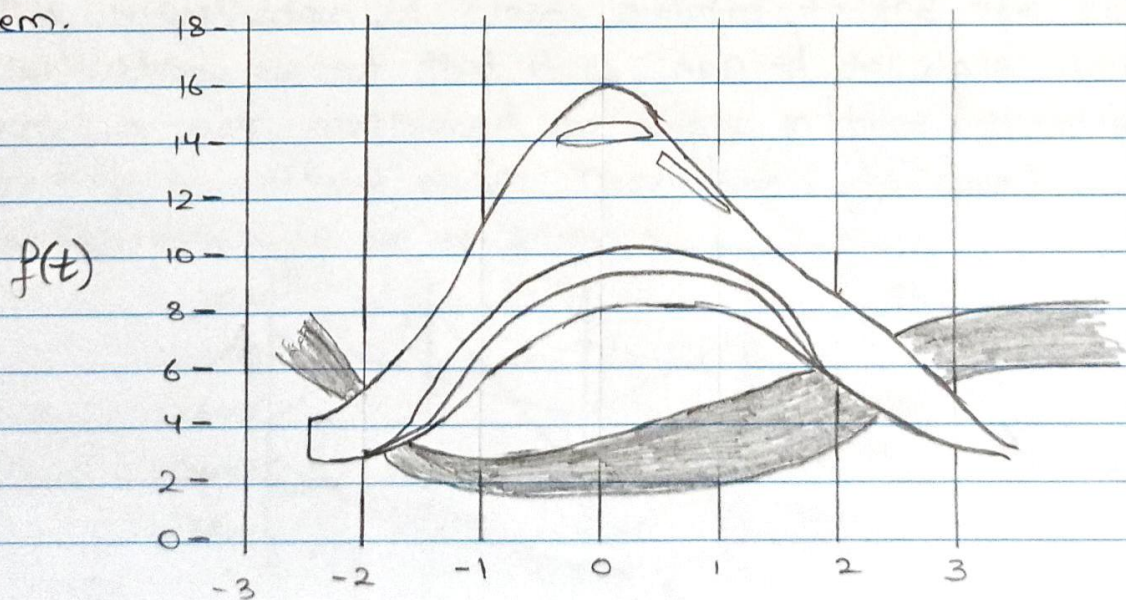
- 1) Andrews curves, also known as Andrews plot are a visualisation technique used to identify underlying structure in multidimensional data. Similar to principal component analysis and T-distributed stochastic neighbor embedding, Andrews curves reduce multiple features into

low dimensional space to display relationships with target variables or classifications. Andrews curves work by mapping all features from a single observation or row of data onto a function. The function is defined as:

$$f_x(\theta) = \frac{x_1}{\sqrt{2} + x_2 \sin(\theta) + x_3 \cos(\theta) + x_4 \sin(2\theta) + x_5 \cos(2\theta)}$$

$f(x)$ represents the sum of function outputs, while $x_1, x_2, x_3, x_4, x_5, x_n$ represent n features or columns of a single observation or row of data and corresponding mapping function.

- 2) The resulting function output creates a unique curve, maintaining a relative distance and variance across all features per function coefficient and between observations. Another way to think of Andrews curves may be a transformed version of a parallel coordinates plot. Combined with multiple observations and distinguished by target variable color mappings, Andrews curves highlight potential clusters among classes or separation between them.



3) Example Andrew curves with the Iris dataset where features are mapped on a scale of $-\pi$ to π on the x -axis and the resulting function output values are plotted on the y -axis. Notice the similarity within classes and the disparities between them.

44
109

109

Assignment :- 2

Q. Assignment on data science analysis and visualisation tools - SPSS, Tableau, Knime, Google Analytics, BigML, etc.

→ Data science tools are essential for visualising and extracting insights from data. Different Central to this field are tools that facilitate data analysis and visualisation, enabling professional to extract insights and communicate findings effectively. Following are the tools:-

1) SPSS (Statistical Package for the Social Service) -
SPSS is a statistical analysis tool widely used in academic and industries for research and data management. It provides robust capabilities for analysing complex datasets using advanced statistical techniques.

Features:-

- a) Advanced statistical analysis capabilities
- b) User friendly interface with drag and drop functionality
- c) Supports predictive analysis and hypothesis testing.

Application:-

- a) Social service research to analyze survey data
- b) Healthcare for clinical trials and patient data analysis.

2) Tableau -

Tableau is a leading data visualisation tool that enables users to create interface/interactive dashboards and visual analytics. It is popular for its intuitive interface and ~~and~~ ability to transform

raw data into meaningful visualisations.

Features:-

- Drag and drop interface for creating interactive dashboards
- Integration with various data sources like Excel, SQL databases and cloud platforms.
- Advanced analytics features such as clustering and trend analysis.

Applications:-

- Business intelligence to track KPIs and visualise sales performance.
- Marketing analytics to identify customer behaviours and trends

3) KNIME -

KNIME (Konstanz Information Miner) is an open source platform designed for data integration, processing, analysis, and visualisation. It allows users to build modular workflows using drag and drop nodes.

~~Features~~ Features:-

- Modular workflow design with drag and drop nodes.
- Support machine learning, text mining and big data analytics.
- Seamless integration with Python, R and other tools.

Applications:-

- Pharmaceutical research for drug discovery using predictive models.
- Financial services for fraud detection and risk assessment.
- Marketing campaigns to analyze customer journeys.

4) Google Analytics -

Google Analytics is a web based tool designed to track website traffic and user behavior. It provides insights into how users interact with websites or apps in real time.

Features:-

- Real time tracking of user interactions on websites or apps.
- Customizable dashboards to monitor metrics like bounce rate, session duration, etc.
- Integration with Google Ads for campaign performance analysis.

Applications:-

- E-commerce business to optimise conversion ~~and~~ rates.
- Digital marketing agencies to measure campaign success.
- Content creators to understand audience engagement patterns.

5) Big ML -

Big ML is a machine learning platform focused on making predictive modeling accessible through ~~attention~~ automation and user friendly interfaces. It simplifies complex workflows in machine learning projects.

Features:-

- Provides tools for classification, regression, clustering, anomaly detection, etc.
- Offers a user friendly interface along with API support.
- Enables one click deployment of predictive models.

Applications:-

- a) Retail businesses for demand forecasting.
- b) Insurance companies for risk modelling.
- c) Education sector for personalised learning experiences.

Each tool - SPSS, Tableau, KNIME, Google Analytics and BigML serves unique purpose in data science.
SPSS → excels in statistical analysis.

Tableau → visualisation

KNIME → Flexibility in workflow automation.

Google Analytics → specialised in web analytics.

BigML → Focuses on Machine Learning

Together these tools empower professionals to derive actionable insights from data efficiently.